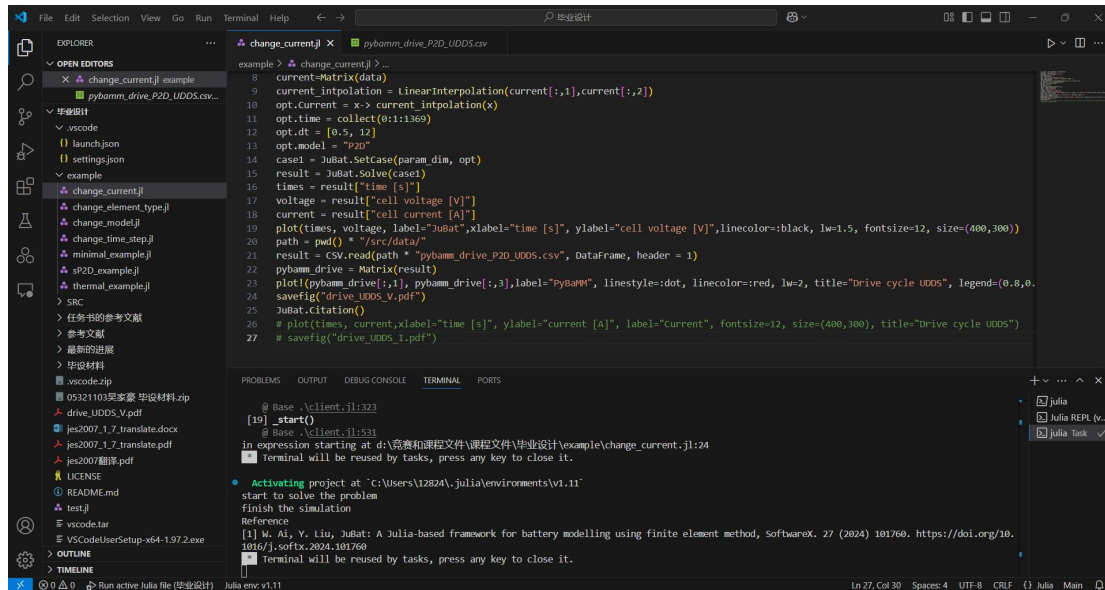


目前可以确定所有的代码都可以运行。

1. change_current

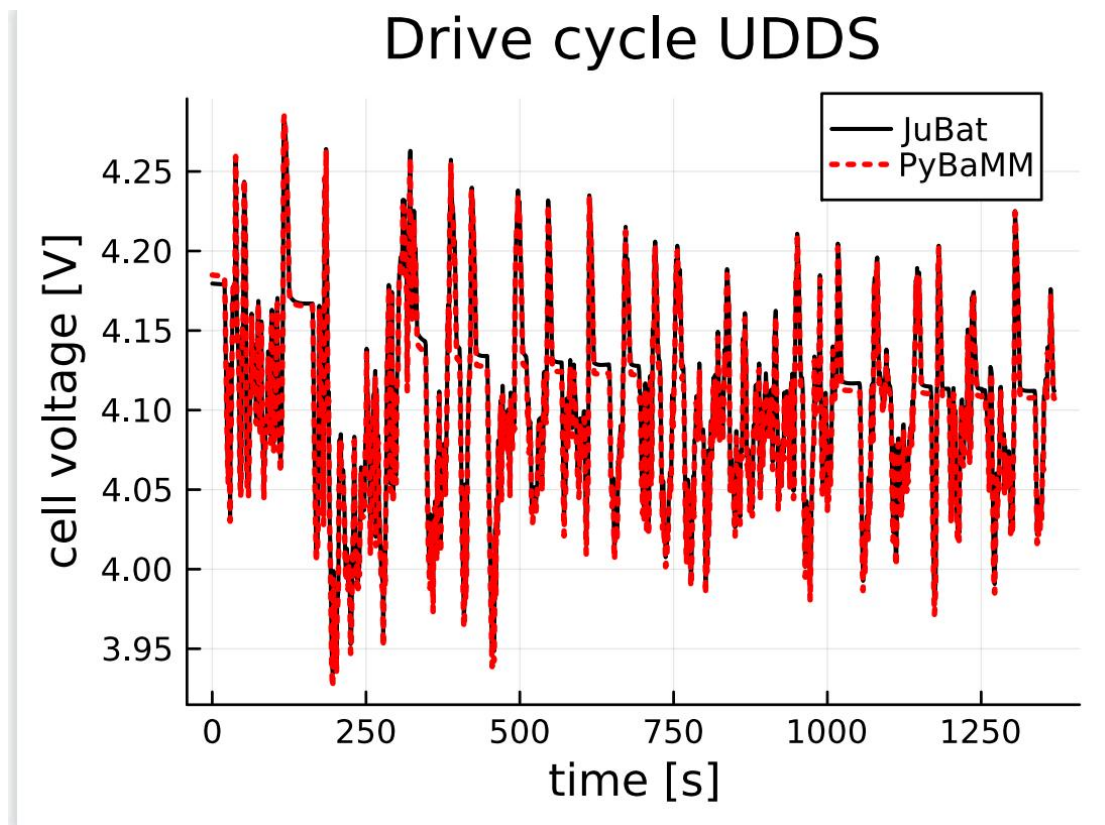


```
example > julia change_current.jl
1 current=Matrix{data}
2 current_interp = linearInterpolation(current[:,1],current[:,2])
3 opt.Current = x-> current_interp(x)
4 opt.time = collect(0:1:1360)
5 opt.dt = [0.5, 12]
6 opt.model = "P2D"
7 case1 = JuBat.SetCase(param_dim, opt)
8 result = JuBat.Solve(case1)
9 times = result["time [s]"]
10 voltage = result["cell voltage [V]"]
11 current = result["cell current [A]"]
12 plot(times, voltage, label="JuBat", xlabel="time [s]", ylabel="cell voltage [V]", linecolor=:black, lw=1.5, fontsize=12, size=(400,300))
13 path = pwd() * "/src/data/"
14 result = CSV.read(path * "pybamm_drive_P2D_UDDS.csv", Dataframe, header = 1)
15 pybamm_drive = Matrix(result)
16 plot!(pybamm_drive[:,1], pybamm_drive[:,3], label="pyBaMM", linestyle=:dot, linecolor=:red, lw=2, title="Drive cycle UDDS", legend=(0.8,0.))
17 savefig("drive_UDDS.V.pdf")
18 JuBat.Citation()
19 # plot(times, current, xlabel="time [s]", ylabel="current [A]", label="Current", fontsize=12, size=(400,300), title="Drive cycle UDDS")
20 # savefig("drive_UDDS.I.pdf")
```

Terminal output:

```
@ Base .\client.jl:323
[19] _start()
@ Base .\client.jl:531
in expression starting at: d:\竞赛和课程文件\课程文件\毕业设计\example\change_current.jl:24
Terminal will be reused by tasks, press any key to close it.

Activating project at "C:\Users\12824\julia\environments\v1.11"
start to solve the problem
finish the simulation
Reference
[1] M. Ai, Y. Liu, JuBat: A Julia-based framework for battery modelling using finite element method, SoftwareX. 27 (2024) 101760. https://doi.org/10.1016/j.softx.2024.101760
Terminal will be reused by tasks, press any key to close it.
```



该图显示的是 UDDS（Urban Dynamometer Driving Schedule）驱动循环下的电池电压变化。

横轴表示时间，纵轴表示电池电压。

- JuBat 曲线（黑色实线）：代表使用 JuBat 仿真工具得到的电池电压随时

间的变化。

- PyBaMM 曲线（红色虚线）：代表使用 PyBaMM 仿真工具得到的电池电压随时间的变化。

该图展示了 JuBat 和 PyBaMM 两个电池模型在 UDDS 驱动循环 下的电池电压对比。尽管两个模型的电压曲线非常接近，但它们在某些时刻的电压值有所不同，可能是由于模型假设、算法或初始条件的不同所致。

2. change_elements_type

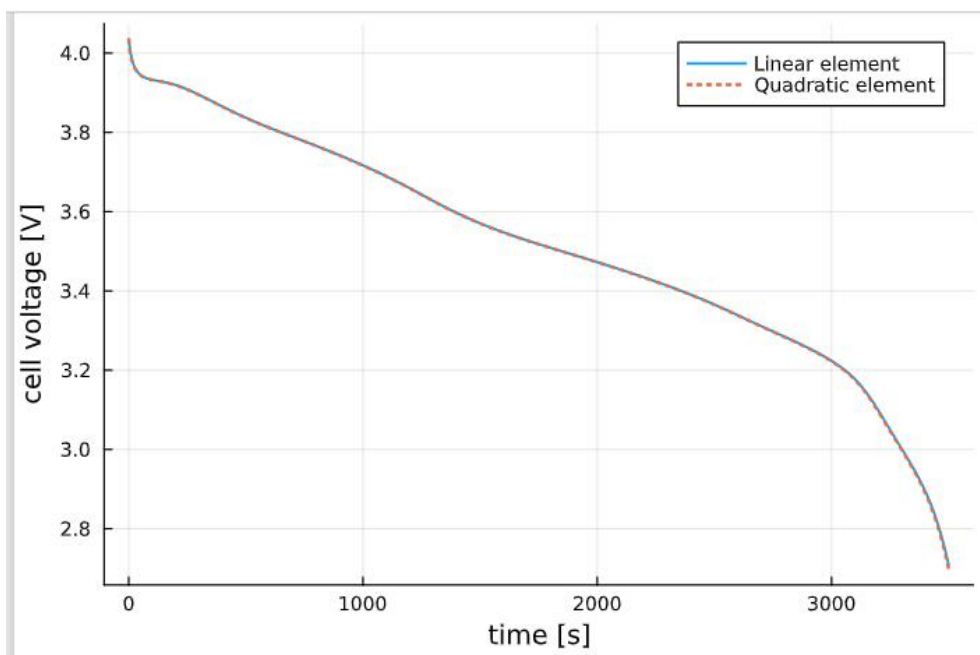
- 代码选择了一个 LG M50 型号的电池模型 (`param_dim = JuBat.ChooseCell("LG M50")`), 并初始化了仿真选项 (`opt = JuBat.Option()`).
- 电流设置为常数 $i = 5$ (恒定电流)。
- 设置了仿真时间 `opt.time = collect(0:100:abs(5/i)*3500)`, 时间步长是 100 秒, 时间范围大约从 0 到 3500 秒。
- 模型为 "P2D" (二维平面模型), 并设置了网格类型 (`meshType`), 分别为 L2 (线性元素) 和 L3 (二次元素)。

• 两个不同网格类型的仿真:

- 第一个仿真使用了 线性元素 (L2), 并保存了结果 `result`。
- 第二个仿真使用了 二次元素 (L3), 并保存了结果 `result1`。

• 绘图:

- 使用 `plot()` 绘制了 线性元素 (L2) 的电池电压随时间变化的曲线, 并设置了标签为 "Linear element"。
- 使用 `plot!()` 在同一张图上叠加了 二次元素 (L3) 的电池电压随时间变化的曲线, 并设置了标签为 "Quadratic element", 并使用虚线样式。



标题: Drive cycle UDDS

- 这张图表展示了电池电压随时间的变化，比较了 **线性元素 (L2)** 和 **二次元素 (L3)** 对电池电压的影响。

横轴 (X 轴): time [s]

- 横轴表示仿真时间，单位为秒 (s)，范围大约从 0 到 3000 秒。

纵轴 (Y 轴): cell voltage [V]

- 纵轴表示电池电压，单位为伏特 (V)，电压从 4.0V 开始，逐渐降低到接近 2.8V。

曲线:

- **蓝色实线 (Linear element)**: 代表使用 **线性元素 (L2)** 网络的仿真结果。电池电压从大约 4.0V 开始，逐渐下降，表现出较平稳的衰减趋势。
- **红色虚线 (Quadratic element)**: 代表使用 **二次元素 (L3)** 网络的仿真结果。电池电压的下降趋势与线性元素非常相似，但有轻微的不同，二次元素的结果在某些时刻可能略有差异，表现出较细腻的曲线形状。

图例:

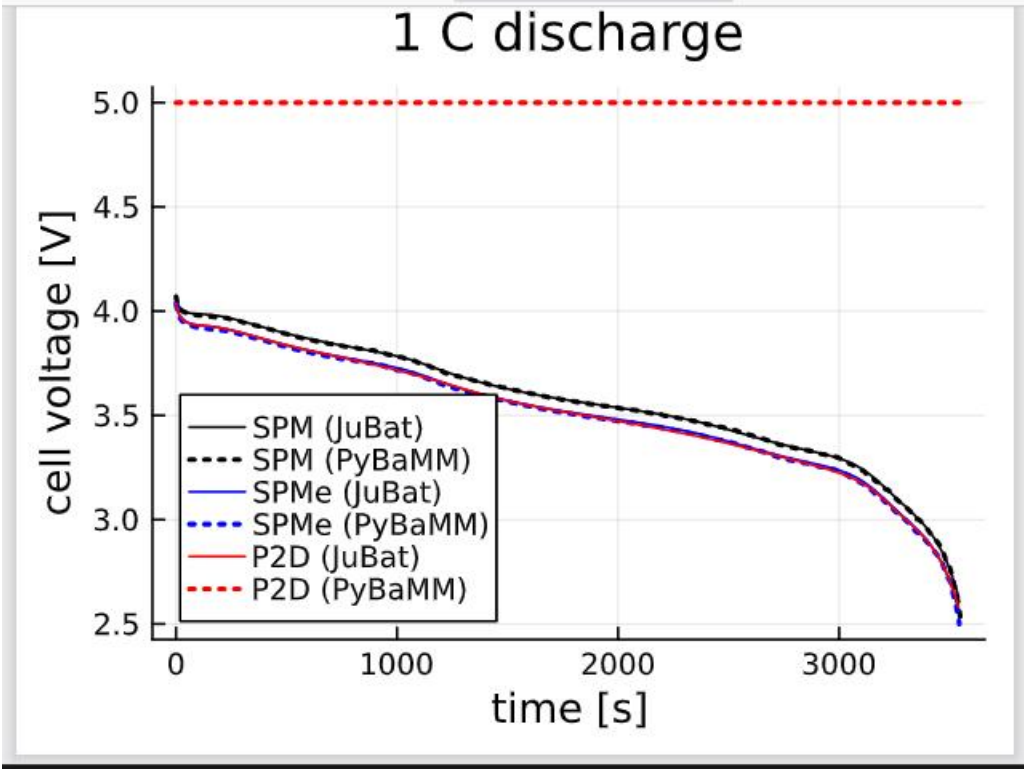
- 图例表明蓝色线条代表 **线性元素 (L2)**，红色虚线代表 **二次元素 (L3)**。

结论:

该图表显示了 **线性元素** 和 **二次元素** 在电池模型仿真中的电压变化对比。尽管两者的变化趋势非常相似，但二次元素模型可能提供了更加平滑或精细的电压衰减曲线。使用二次元素（L3）可能会提供更高的精度和更复杂的数值解。

3. change_model

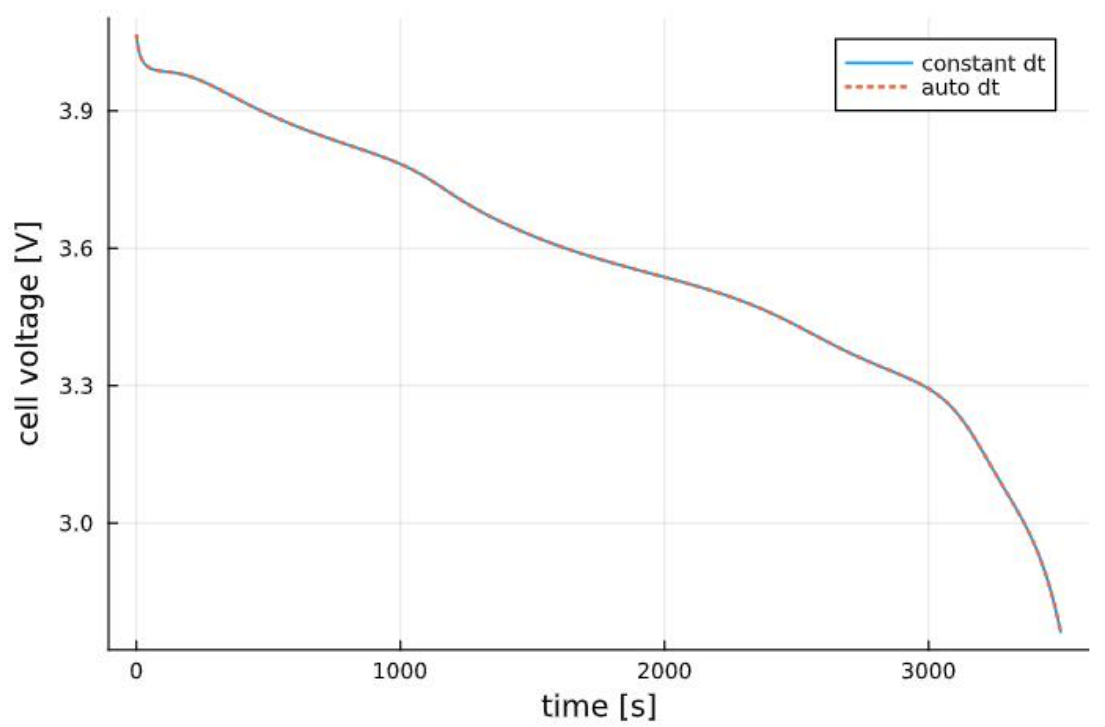
固定电流大小，电池型号，对比 **JuBat** 和 **PyBaMM** 工具的不同电池模型（SPM, SPMe, P2D）在 **1C 放电速率** 下的电压变化，展示这两款仿真工具在不同模型下的仿真结果。通过这张图，可以评估不同模型和仿真工具的精度差异。



- **标题:** 1 C discharge
 - 该图显示的是电池在 **1C 放电速率** 下的电压变化。
- **横轴 (X 轴):** time [s]
 - 横轴表示仿真时间, 单位为秒 (s), 范围大约从 0 到 3000 秒。
- **纵轴 (Y 轴):** cell voltage [V]
 - 纵轴表示电池电压, 单位为伏特 (V), 电压从大约 4.0V 开始, 并逐渐降低至接近 2.5V。
- **曲线:**
 - **黑色实线** (SPM (JuBat)): 代表使用 **SPM** (单粒子模型) 进行 JuBat 仿真得到的电池电压随时间变化。
 - **黑色虚线** (SPM (PyBaMM)): 代表使用 **PyBaMM** 仿真工具得到的 **SPM** 模型电池电压随时间变化。
 - **蓝色实线** (SPMe (JuBat)): 代表使用 **SPMe** (增强的单粒子模型) 进行 JuBat 仿真得到的电池电压随时间变化。
 - **蓝色虚线** (SPMe (PyBaMM)): 代表使用 **PyBaMM** 仿真工具得到的 **SPMe** 模型电池电压随时间变化。
 - **红色实线** (P2D (JuBat)): 代表使用 **P2D** (二维模型) 进行 JuBat 仿真得到的电池电压随时间变化。
 - **红色虚线** (P2D (PyBaMM)): 代表使用 **PyBaMM** 仿真工具得到的 **P2D** 模型电池电压随时间变化。

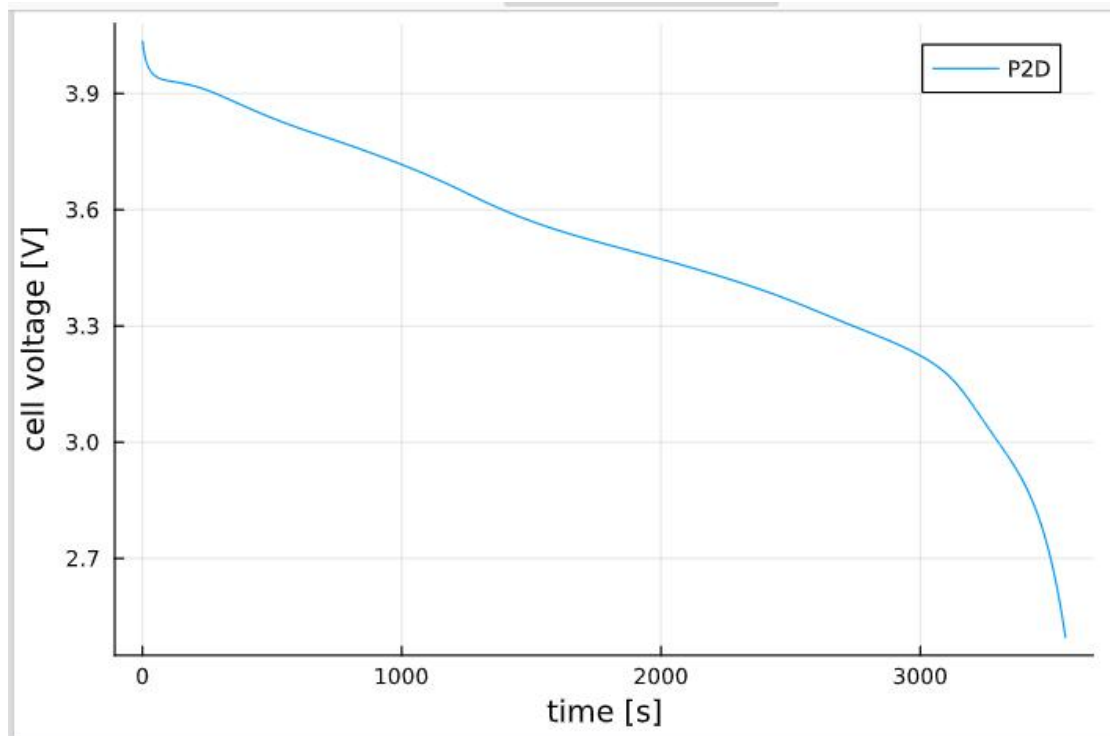
对于 SPM 和 SPMe, JuBat 模型的仿真结果和 PyBaMM 非常接近。但是对于 P2D 模型, Jubat 的结果仍然与前面的结果曲线相近, 而 PyBaMM 结果相差非常大。可能的原因是相比其它模型, P2D 模型更复杂, 涉及到电池的空间分布、浓度极化、温度效应等多个因素。每个仿真工具在实现 P2D 模型时可能会有不同的数值解法或近似, 尤其是在处理复杂的多维问题时, 结果的差异可能更为显著。

4.change_time steps



- **蓝色实线** (constant dt)：表示使用 **常规时间步长** 的仿真结果。电池电压随时间逐渐下降，表现为平滑的曲线。
- **红色虚线** (auto dt)：表示使用 **自适应时间步长** 的仿真结果。该曲线也表现为电池电压随时间的下降，形状非常接近常规时间步长的结果，但在仿真过程中，自动调整的时间步长可能使得曲线略微不同。
- **常规时间步长** (constant dt)：设置了仿真使用常规的时间步长。
- **自适应时间步长** (auto dt)：设置了仿真使用自适应时间步长，`opt.dtType = "auto"`，并指定了阈值 `opt.dtThreshold = 0.0001`，表示仿真过程中时间步长会根据误差自动调整。

5.minimal_example



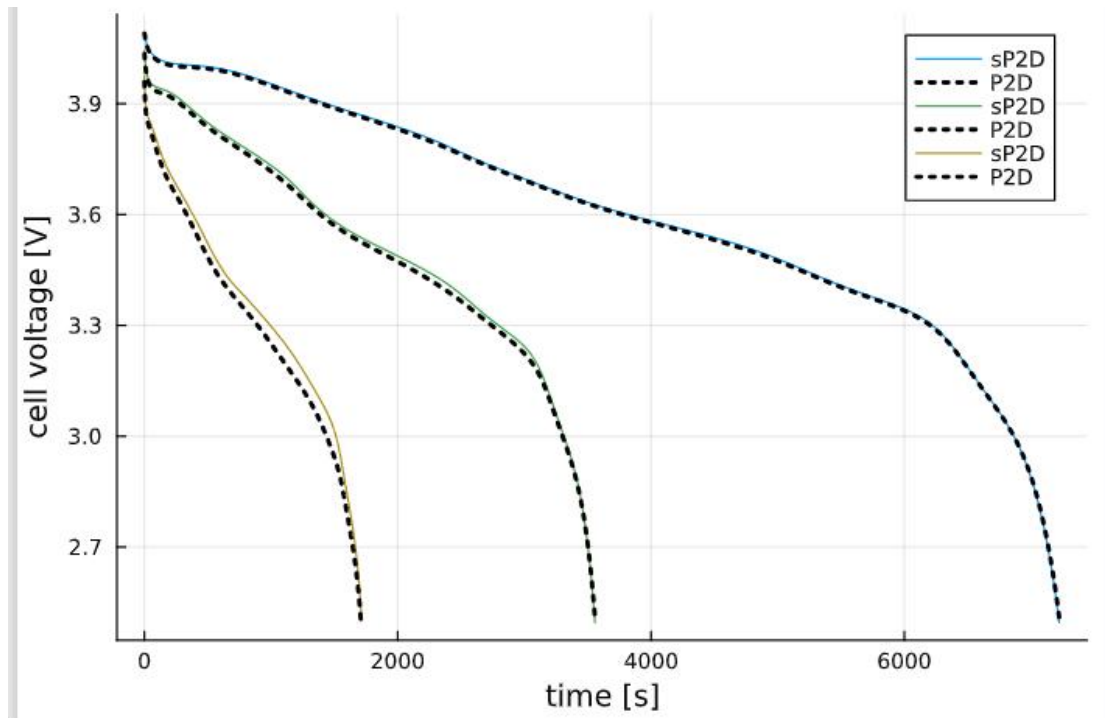
这张图展示了 **P2D 模型** 在 **1C 放电速率** 下电池电压随时间变化的过程，电池电压逐渐下降，符合实际电池放电过程的预期。使用 **P2D 模型** 进行仿真能够较为精确地预测电池的性能。

- **电池电压衰减**: 从图中可以看出，电池电压随时间逐渐下降。这是典型的 **放电过程**，电池的电压会随着电流的流动和能量的释放而逐步减少。
- **P2D 模型**: 使用 **P2D 模型** 可以更精确地模拟电池的行为，尤其是在考虑到浓度极化、电荷传输等因素时，该模型提供了比简单的 **SPM** 或 **SPMe** 模型更为复杂和精细的电池行为描述。
- **放电曲线**: 从电压衰减的形状来看，电池的电压下降趋势是平缓的，直到最后快速下降，反映了电池放电过程中电压的自然衰减。

6. Sp2d_example

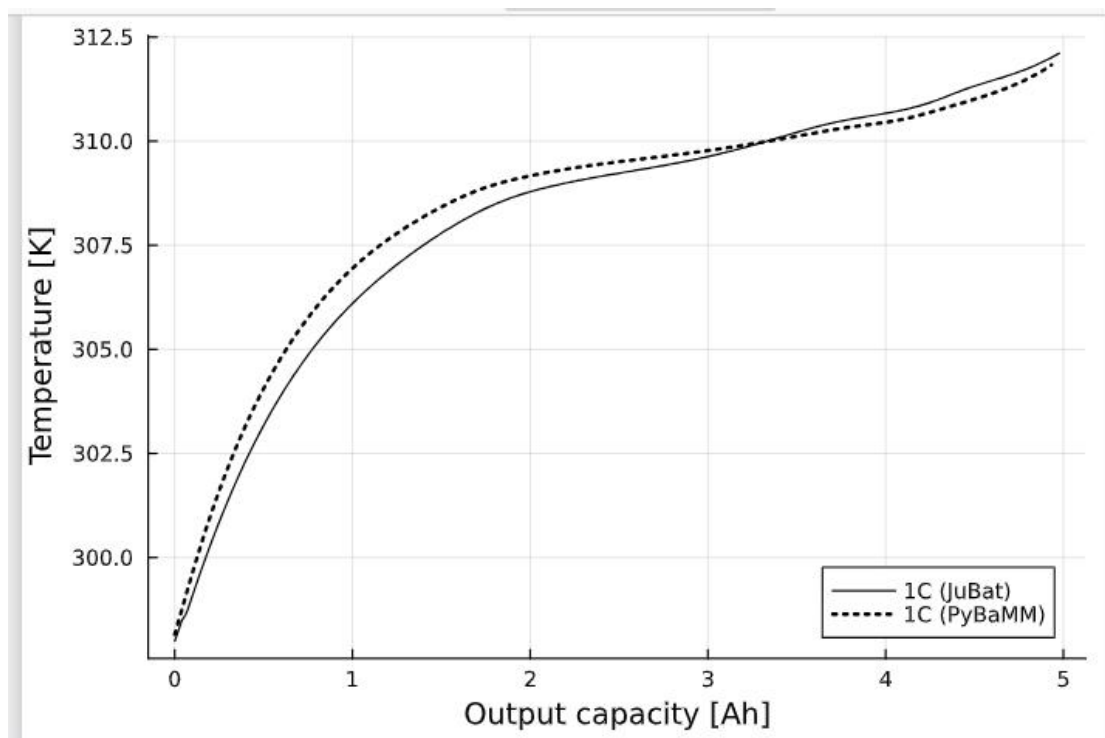
这张图展示了 **sP2D** 和 **P2D** 模型在不同放电速率下电池电压变化的对比。定义了不同的放电速率 ($\text{rates} = [0.5, 1, 2]$)。电池电压随时间的推移逐渐下降，在较高放电速率（如 2C）下，电池电压下降的速度较快。黄色：2c，绿色：1c，蓝色：0.5c

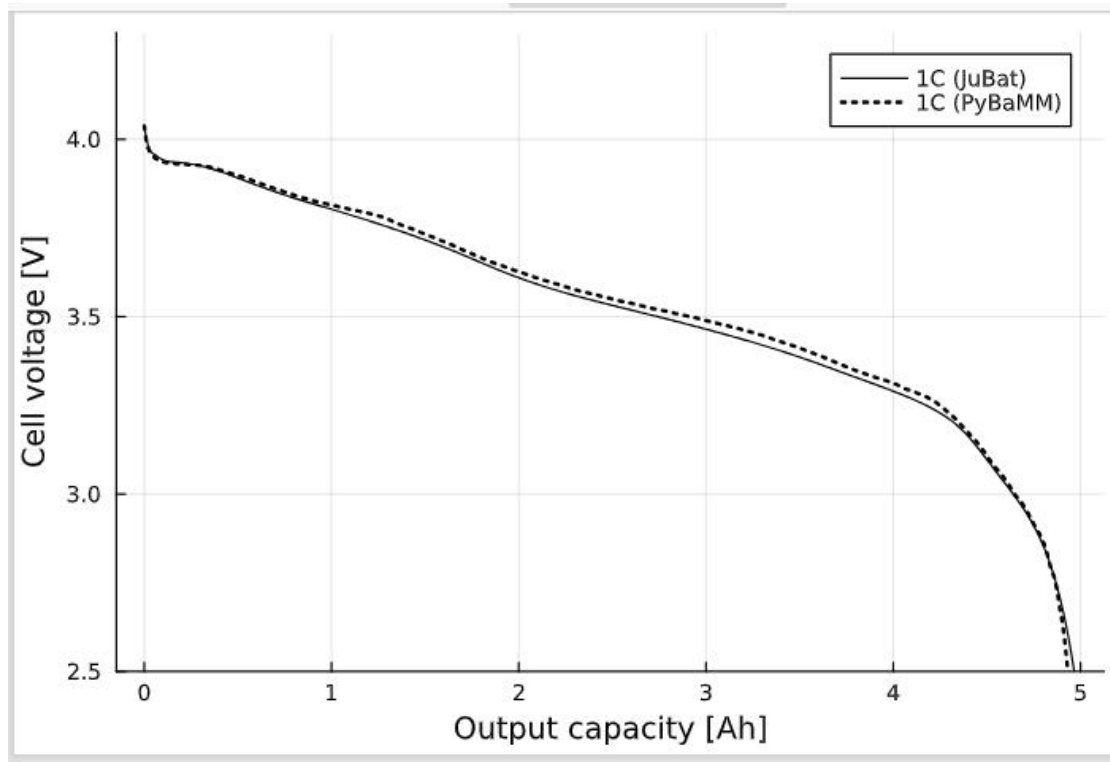
可以看出来结果是没有什么明显区别的。



7. thermal_example

这些图表展示了 **JuBat** 和 **PyBaMM** 在电池电压和温度方面的仿真结果对比，验证了两种仿真工具在相同电池模型下的表现。两者的结果非常接近，说明在这些条件下，它们对电池行为的预测具有较高的一致性。





- **pV**: 绘制了 电池电压 随 输出容量 (Ah) 变化的曲线。
- **pT**: 绘制了 电池温度 随 输出容量 (Ah) 变化的曲线。
- 两种模型 (**JuBat** 和 **PyBaMM**) 的结果分别用实线和虚线表示。

对于每个 放电速率 (Crate)，分别运行 **JuBat** 和 **PyBaMM** 仿真，计算了 电池电压 (Cell voltage) 和 温度 (Temperature) 随时间的变化。

在相同的 放电速率 (1C) 下，**JuBat** 和 **PyBaMM** 的电池电压随时间的变化趋势非常相似，但两者在数值上可能存在小的差异。这可能是由于两者模型细节或求解方法的不同。

对于 电池温度，两者的变化趋势也非常接近，表明 **JuBat** 和 **PyBaMM** 在温度建模方面提供了相似的结果。

8. Mechanical_example

报错了：

```

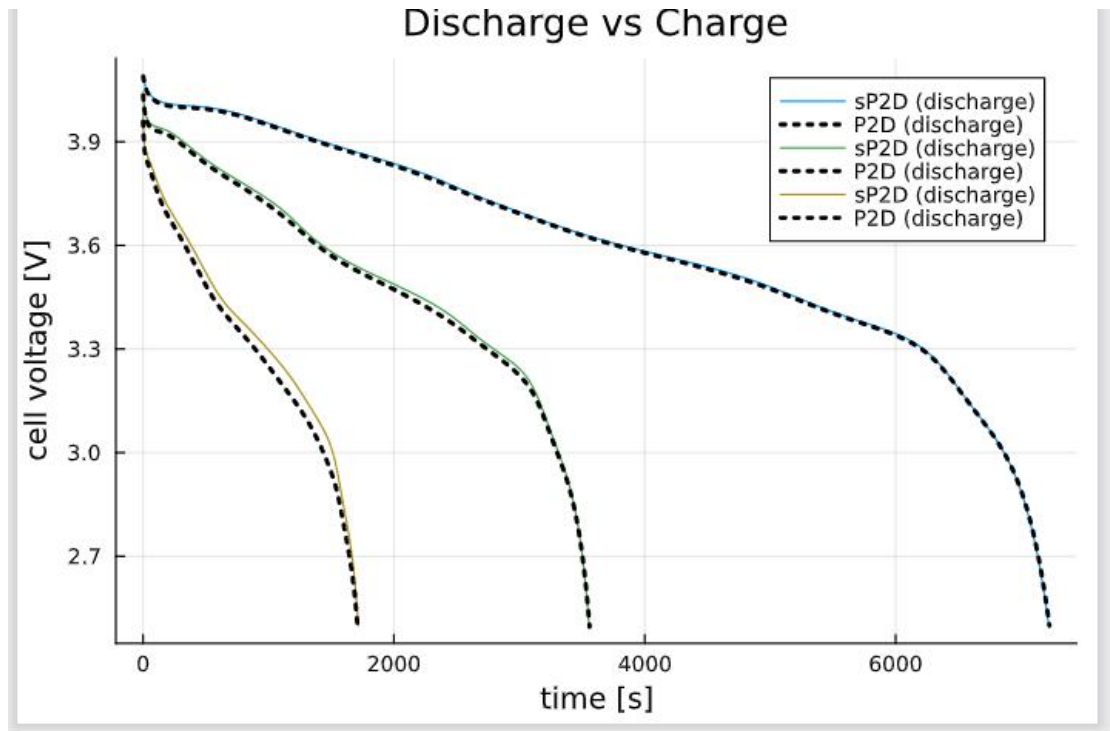
• Activating project at `C:\Users\12824\.julia\environments\v1.11`
ERROR: LoadError: type Option has no field mechanicalmodel
Stacktrace:

```

Why? 没库？

任务书目标实现：

一、利用 JuBat 软件模拟电池在不同倍率恒定电流下的充放电
放电：



充电：

失败案例一：

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
rates = [0.5, 1, 2]
ts = [3700, 3700, 3600, 3600]

# 创建一个图形进行放电和充电的对比
pV_discharge_charge = plot(xlabel="time [s]", ylabel="cell voltage [V]", title="Discharge vs
Charge", lw=1)

for j = 1:3
    Crate = rates[j]
    opt.dt = [1, 20] / Crate
```

```

opt.dtType = "fixed"

# 放电仿真
i = 5 * Crate
opt.Current = x -> i # 负电流表示放电
opt.time = [0 ts[j] / Crate]
opt.model = "sP2D"
case = JuBat.SetCase(param_dim, opt)
result = JuBat.Solve(case)

```

```

# 放电仿真 P2D
opt.model = "P2D"
case2 = JuBat.SetCase(param_dim, opt)
result2 = JuBat.Solve(case2)

```

```

# 绘制放电结果
plot!(pV_discharge_charge, result["time [s]"], result["cell voltage [V]"], label="sP2D
(discharge)", lw=1)

plot!(pV_discharge_charge, result2["time [s]"], result2["cell voltage [V]"], label="P2D
(discharge)", linestyle=:dot, linecolor=:black, lw=2.5)

```

```

# 充电仿真
opt.Current = x -> -i # 正电流表示充电
opt.time = [0 ts[j] / Crate]
case_charge = JuBat.SetCase(param_dim, opt)
result_charge = JuBat.Solve(case_charge)

```

```

# 充电仿真 P2D
opt.model = "P2D"
case_charge2 = JuBat.SetCase(param_dim, opt)
result_charge2 = JuBat.Solve(case_charge2)

```

```

# 绘制充电结果
plot!(pV_discharge_charge, result_charge["time [s]"], result_charge["cell voltage [V]"],
label="sP2D (charge)", lw=1)

plot!(pV_discharge_charge, result_charge2["time [s]"], result_charge2["cell voltage [V]"],
label="P2D (charge)", linestyle=:dot, linecolor=:black, lw=2.5)
end

```

```

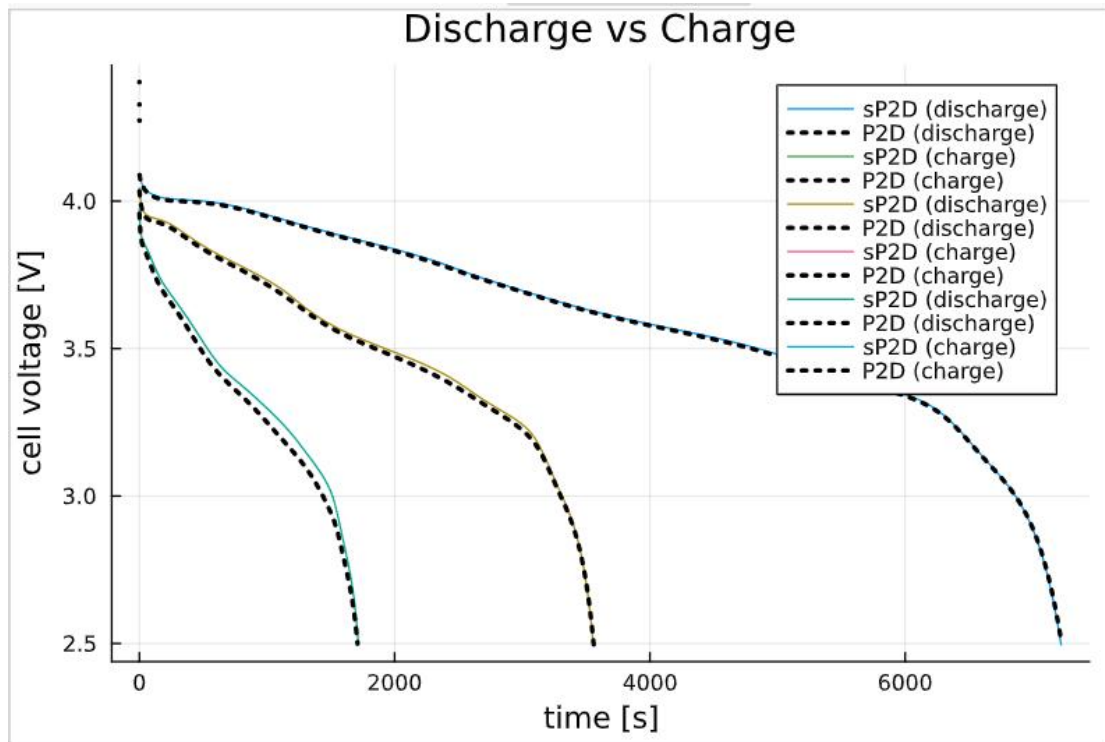
# 保存完整的充放电循环图
savefig(pV_discharge_charge, "discharge_charge_cycle_comparison.pdf")

```

```

JuBat.Citation(["ai2024b"])

```



失败案例二：

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
rates = [0.5, 1, 2]
ts = [3700, 3700, 3600, 3600]

# 为放电和充电分别绘制电压曲线
pV_discharge = plot(xlabel="time [s]", ylabel="cell voltage [V]", title="Discharge vs Charge",
lw=1)
pV_charge = plot(xlabel="time [s]", ylabel="cell voltage [V]", title="Discharge vs Charge", lw=1)

for j = 1:3
    Crate = rates[j]
    opt.dt = [1, 20] / Crate
    opt.dtType = "fixed"

    # 放电仿真
    i = 5 * Crate
    opt.Current = x -> i # 负电流表示放电
    opt.time = [0 ts[j] / Crate]
    opt.model = "sP2D"
    case = JuBat.SetCase(param_dim, opt)
```

```
result = JuBat.Solve(case)
```

```
# 放电仿真 P2D
opt.model = "P2D"
case2 = JuBat.SetCase(param_dim, opt)
result2 = JuBat.Solve(case2)
```

```
# 绘制放电结果
plot!(pV_discharge, result["time [s]"], result["cell voltage [V]"], label="sP2D (discharge)",
lw=1)
plot!(pV_discharge, result2["time [s]"], result2["cell voltage [V]"], label="P2D (discharge)",
linestyle=:dot, linecolor=:black, lw=2.5)
```

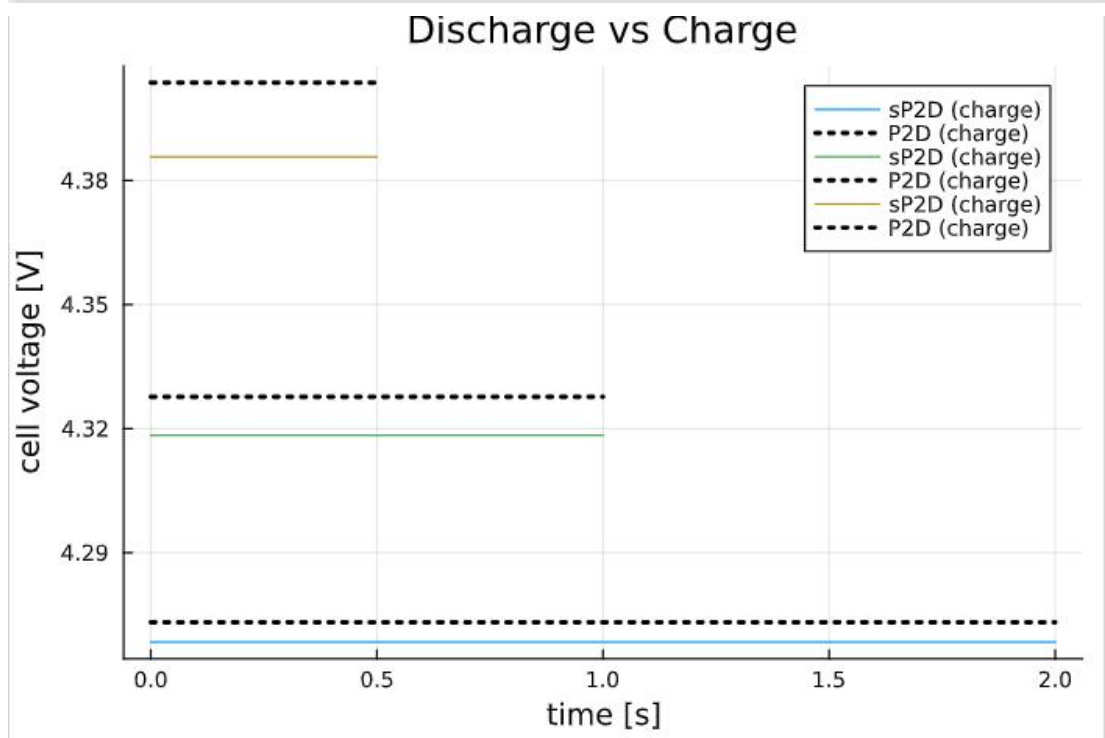
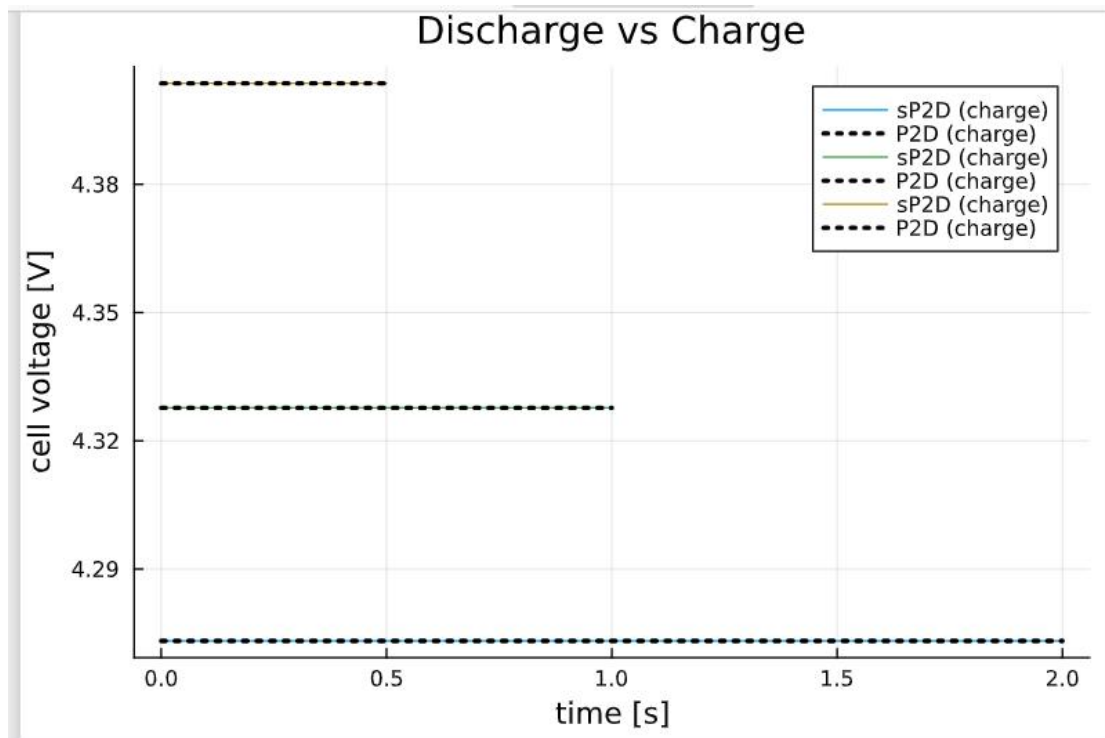
```
# 充电仿真
opt.Current = x -> -i # 正电流表示充电
opt.time = [0 ts[j] / Crate]
case_charge = JuBat.SetCase(param_dim, opt)
result_charge = JuBat.Solve(case_charge)
```

```
# 充电仿真 P2D
opt.model = "P2D"
case_charge2 = JuBat.SetCase(param_dim, opt)
result_charge2 = JuBat.Solve(case_charge2)
```

```
# 绘制充电结果
plot!(pV_charge, result_charge["time [s]"], result_charge["cell voltage [V]"], label="sP2D
(charge)", lw=1)
plot!(pV_charge, result_charge2["time [s]"], result_charge2["cell voltage [V]"], label="P2D
(charge)", linestyle=:dot, linecolor=:black, lw=2.5)
end
```

```
# 保存放电和充电的比较图
savefig(pV_discharge, "discharge_comparison.pdf")
savefig(pV_charge, "charge_comparison.pdf")
```

```
JuBat.Citation(["ai2024b"])
```



失败案例三:

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# Define the rates, time steps for discharging and charging
rates = [0.5, 1, 2]
rates2 = [2.5, 5, 10]
ts_discharge = [3700, 3700, 3600, 3600]
ts_charge = [3700, 3700, 3600, 3600] # Time for charging
```

```
# For discharging simulation
for j = 1:3
    Crate = rates[j]
    opt.dt = [1, 20]/Crate
    opt.dtType = "fixed"
    i = 5 * Crate
    opt.Current = x -> i # Current for discharging
    opt.time = [0 ts_discharge[j]/Crate]
    opt.model = "sP2D"
    case = JuBat.SetCase(param_dim, opt)
    result_discharge = JuBat.Solve(case)
```

```
opt.model = "P2D"
case2 = JuBat.SetCase(param_dim, opt)
result2_discharge = JuBat.Solve(case2)
```

```
# Save the final state after discharge
final_voltage = result_discharge["cell voltage [V]"][-1]
```

```
# Plotting discharging results
plot!(result_discharge["time [s]"], result_discharge["cell voltage [V]"], label="sP2D
Discharge", xlabel="time [s]", ylabel="cell voltage [V]", lw=1)
plot!(result2_discharge["time [s]"], result2_discharge["cell voltage [V]"], label="P2D
Discharge", linestyle=:dot, linecolor=:black, lw=2.5)
```

```
# For charging simulation, reset the battery state to the final voltage after discharge
for charge_rate in rates2
    opt.dt = [1, 20]/Crate
    opt.dtType = "fixed"
    opt.Current = x -> charge_rate # Set current for charging
    opt.time = [0 ts_charge[j]/Crate]

    # Initialize the charge simulation with the final voltage state from discharge
    opt.initial_voltage = final_voltage # Assuming this field is used to set the initial voltage
    opt.model = "sP2D"
    case_charge = JuBat.SetCase(param_dim, opt)
    result_charge = JuBat.Solve(case_charge)
```

```

opt.model = "P2D"

case2_charge = JuBat.SetCase(param_dim, opt)

result2_charge = JuBat.Solve(case2_charge)

```

```

# Plotting charging results

plot!(result_charge["time [s]"], result_charge["cell voltage [V]"], label="sP2D Charge",
linestyle=:dash, lw=2.5)

plot!(result2_charge["time [s]"], result2_charge["cell voltage [V]"], label="P2D Charge",
linestyle=:dashdot, linecolor=:blue, lw=2)

end

end

```

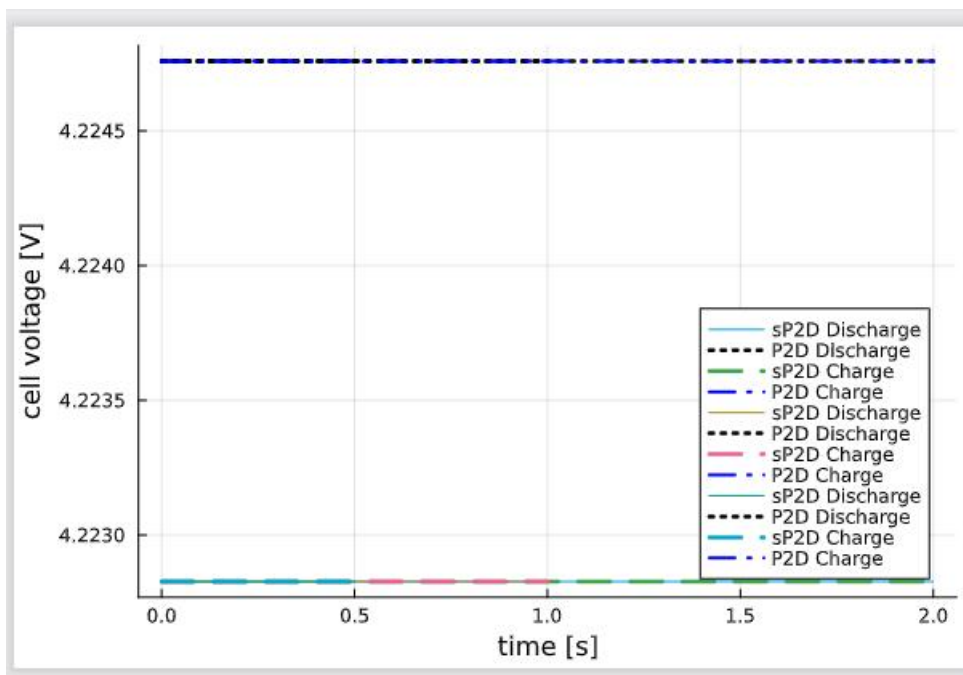
```

# Save the figure as a PDF

savefig("sP2D_and_P2D_voltage_comparison.pdf")

JuBat.Citation(["ai2024b"])

```



成功:

```

using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()

```

```

# 充放电电流参数
charging_current = 4.0 # 充电电流（安培）
discharging_current = -4.0 # 放电电流（安培）

```

```
charge_time = 2000 # 充电时间（秒）
discharge_time = 2000 # 放电时间（秒）
total_time = charge_time + discharge_time # 总时间
```

```
opt.dtType = "fixed"
opt.time = [0, total_time] # 设置总模拟时间
```

```
# 设置电流随时间变化：充电和放电阶段
opt.Current = (t) -> begin
    if t < charge_time
        return charging_current # 充电阶段
    else
        return discharging_current # 放电阶段
    end
end
```

```
# 运行模型模拟
for model in ["sP2D", "P2D"]
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

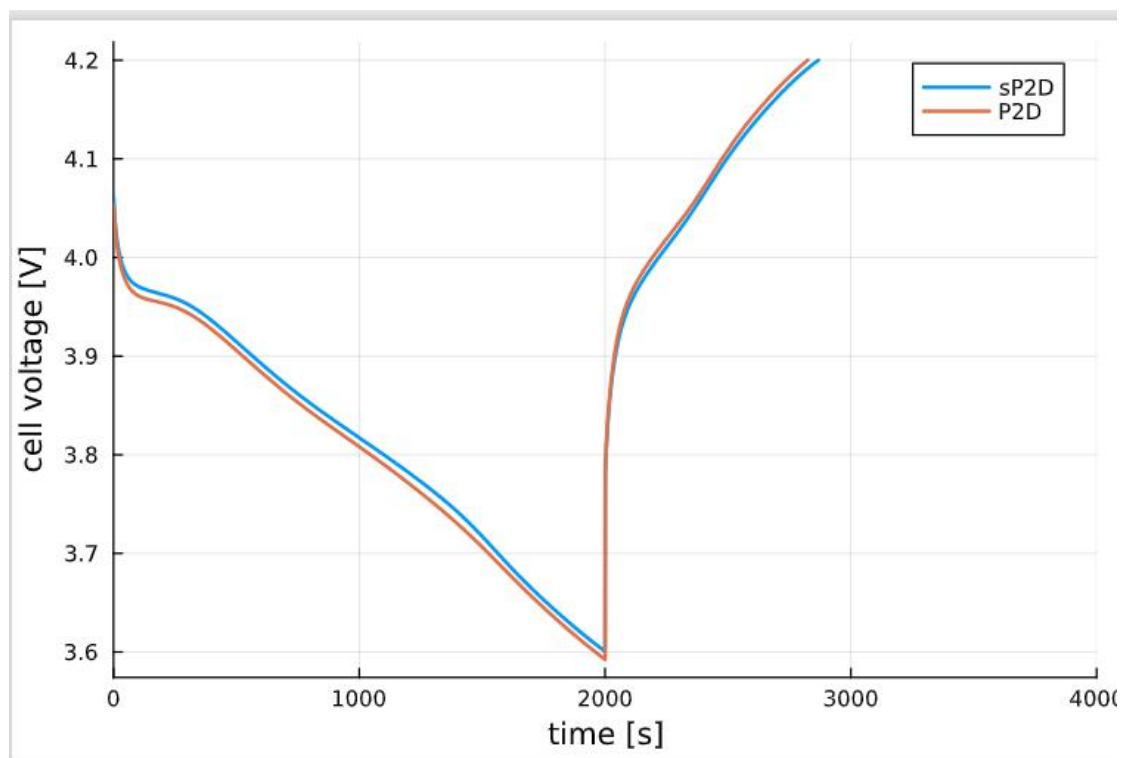
```
    # 绘制结果
    plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="time [s]",
ylabel="cell voltage [V]", lw=2)
end
```

```
# 设置 x 轴显示范围为 total_time
xlims!(0, total_time)
```

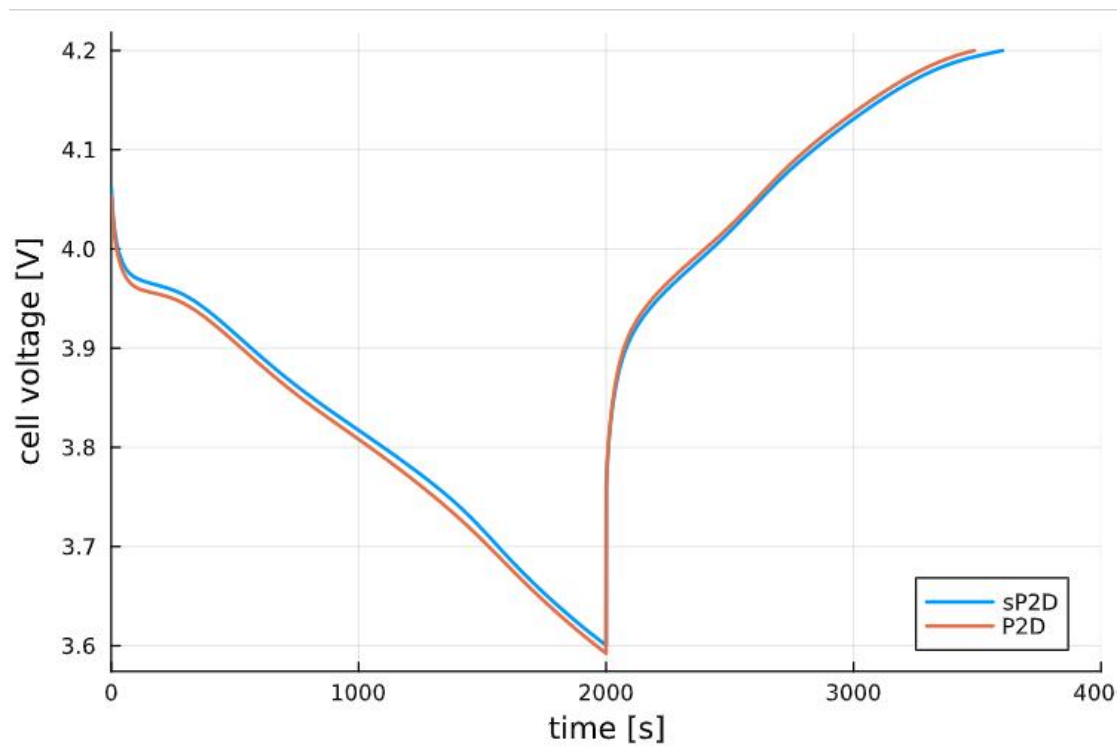
```
# 保存图形
savefig("charge_discharge_current_validation.pdf")
```

```
# 引用
JuBat.Citation(["ai2024b"])
```

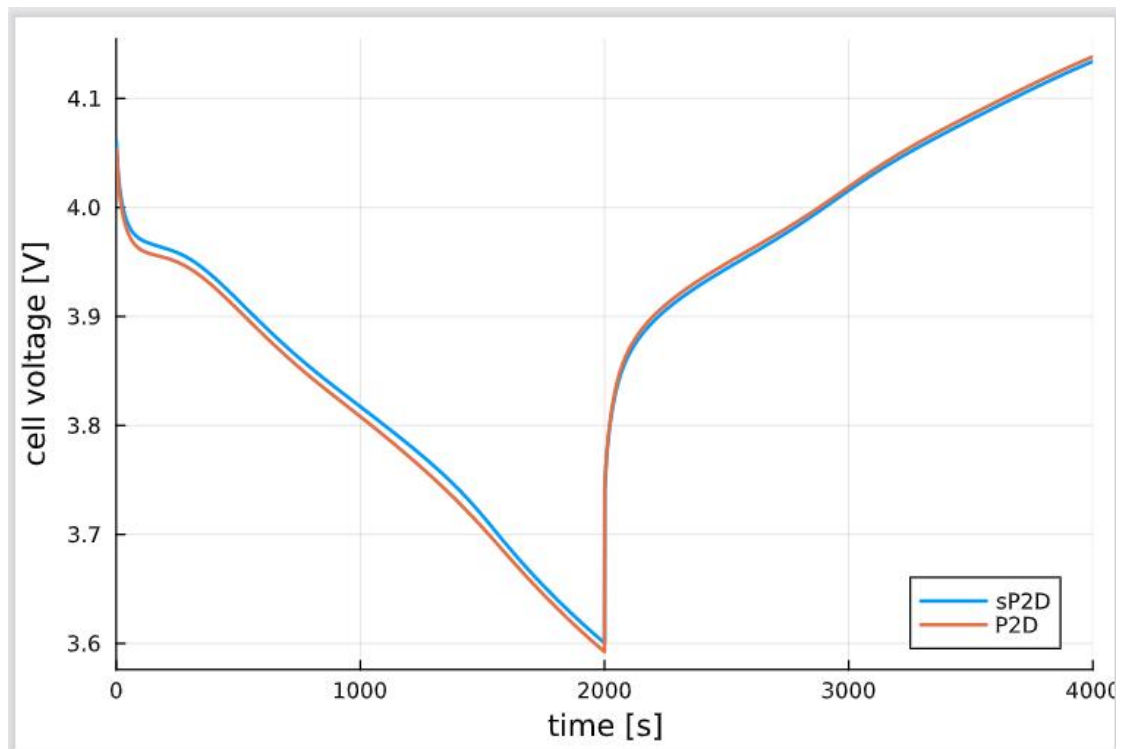
充电电流: -4A:



充电电流: -3A:



充电电流：-2A:



二、阶梯电流

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")
```

```
param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# Define current steps and corresponding times
current_steps = [0.5, 1.0, 2.0, 1.5] # Current steps (A)
time_steps = [0, 1000, 2000, 3000] # Time steps (s) when current changes
```

```
opt.dtType = "fixed"
opt.time = [0, time_steps[end]+1000] # Set total simulation time
```

```
# Function to define the current as a step profile
opt.Current = (t) -> begin
    idx = findfirst(x -> t < x, time_steps)
    idx = isnothing(idx) ? length(current_steps) : idx - 1
    return current_steps[idx]
end
```

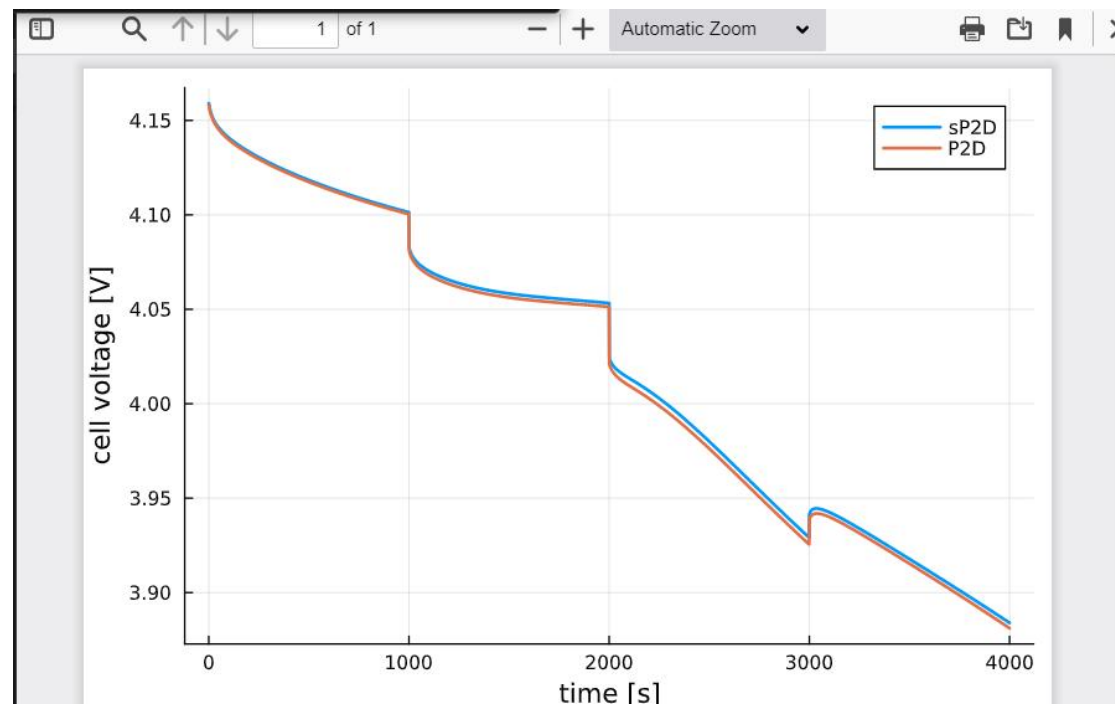
```
# Run simulations for both models
```

```
for model in ["sP2D", "P2D"]
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

```
# Plotting results
plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="time [s]",
ylabel="cell voltage [V]", lw=2)
end
```

```
# Save the plot
savefig("step_current_validation.pdf")
```

```
# Reference for citation
JuBat.Citation(["ai2024b"])
```



三、脉冲电流

(1) 放电工作情况

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# Define pulse current parameters
```

```
pulse_current = 2.0    # Peak pulse current in Amperes
low_current = 0.5      # Low current in Amperes
pulse_width = 200      # Pulse width in seconds
pulse_interval = 1000  # Pulse interval in seconds
total_time = 3000      # Total simulation time in seconds
```

```
opt.dtType = "fixed"
opt.time = [0, total_time] # Set total simulation time
```

```
# Function to define the pulse current
opt.Current = (t) -> begin
    # Determine if the current is in a pulse period
    if t % (pulse_width + pulse_interval) < pulse_width
        return pulse_current # Apply pulse current during pulse period
    else
        return low_current # Apply low current outside pulse period
    end
end
```

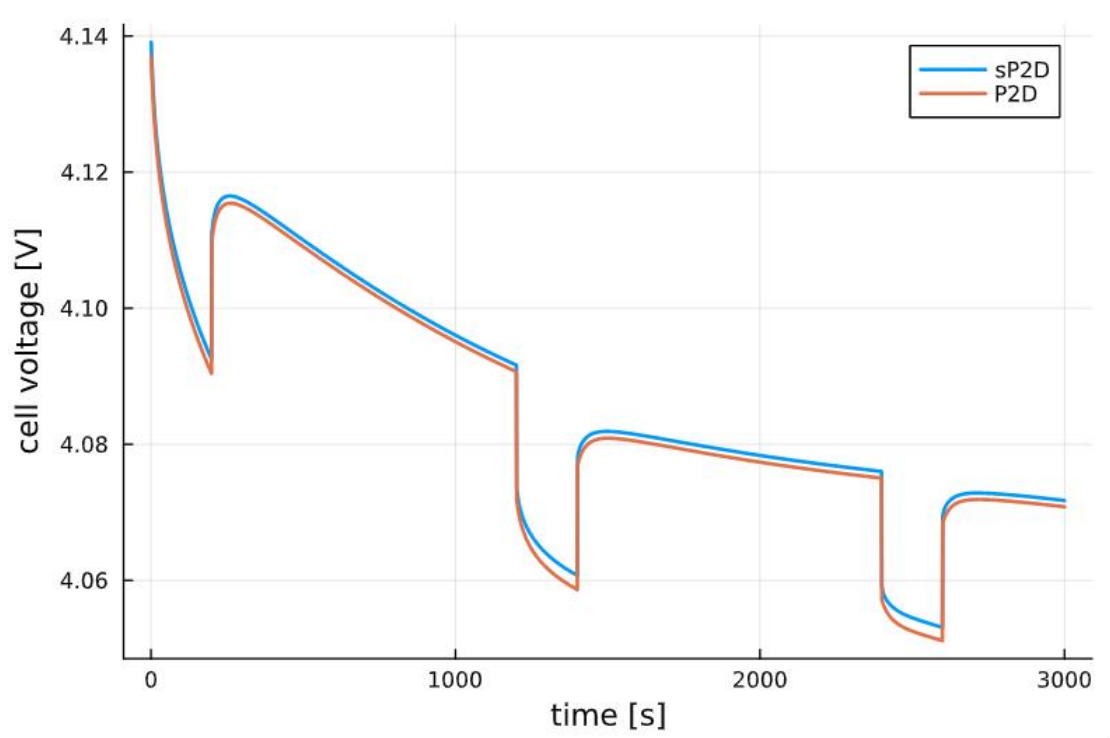
```
# Run simulations for both models
for model in ["sP2D", "P2D"]
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

```
    # Plotting results
    plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="time [s]",
ylabel="cell voltage [V]", lw=2)
end
```

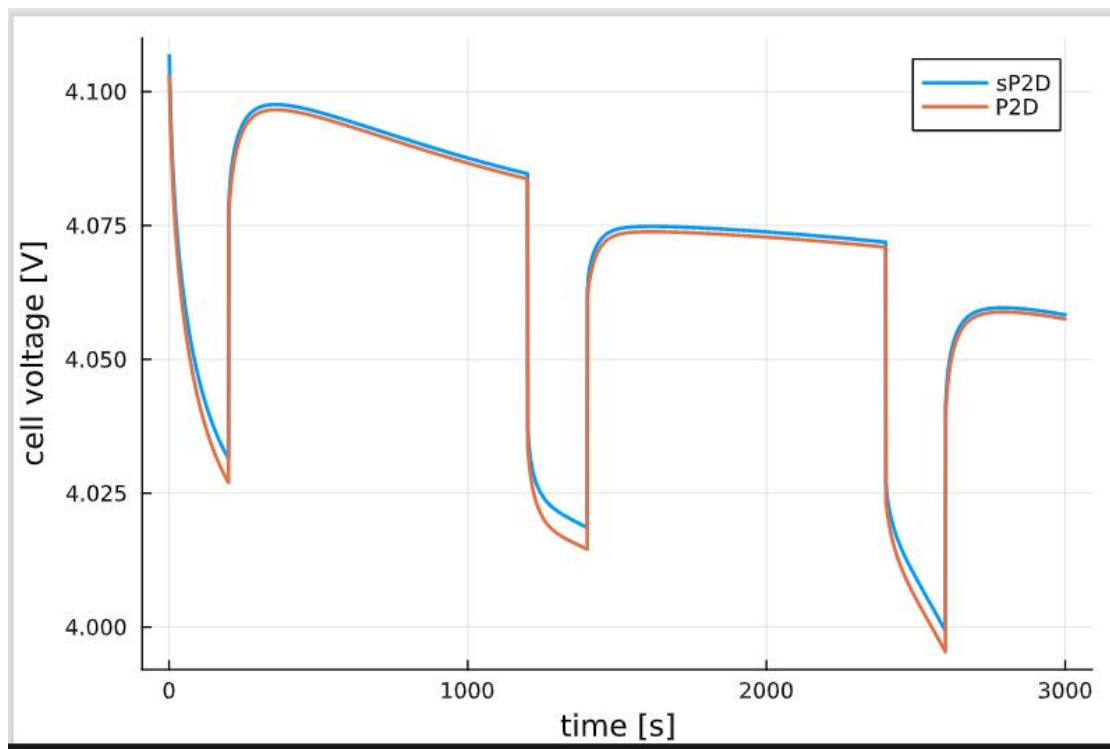
```
# Save the plot
savefig("pulse_current_validation.pdf")
```

```
# Reference for citation
JuBat.Citation(["ai2024b"])
```

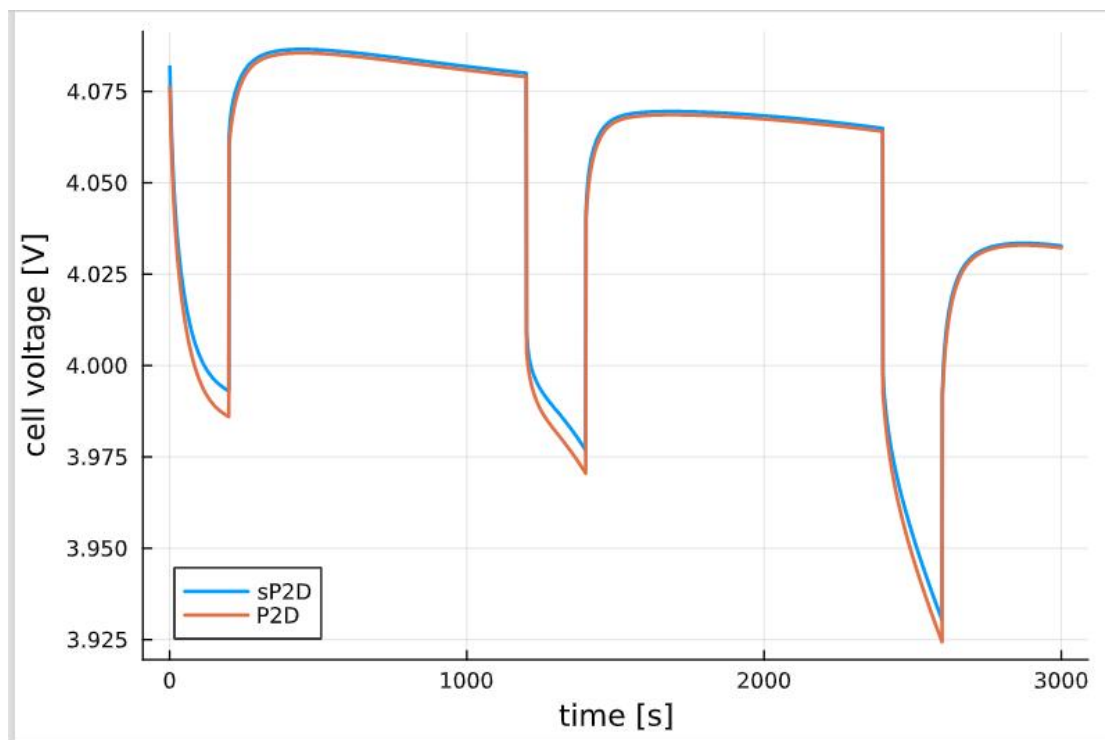
峰值: 1.0A:



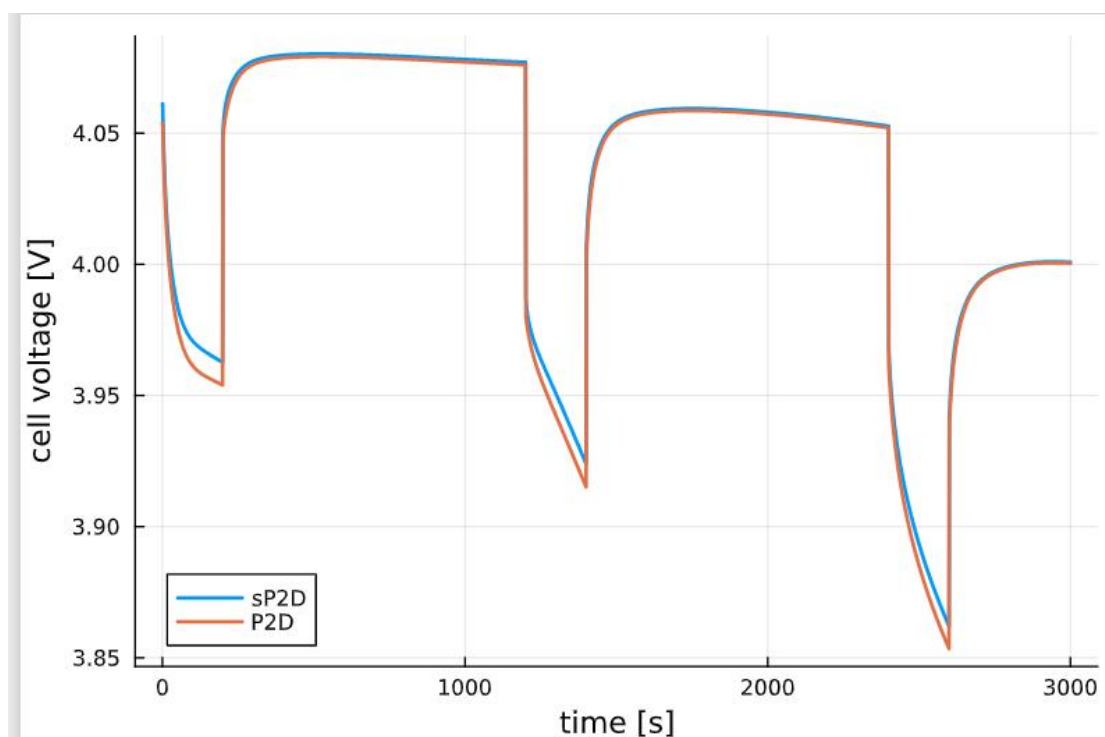
峰值: 2.0A:



峰值: 3.0A:



峰值: 4.0A:



现象：在脉冲电流的放电过程中，电压的变化呈现出一种不太寻常的模式。在电流峰值从 1A 增加到 2A 时，电压下降幅度加大，符合预期。然而，当电流继续增大到 3A、4A 时，电压下降幅度反而变得更小，这是由于几个因素的综合作用。

主要原因分析：

电池的内阻：电池的内阻是影响电池放电电压的一个重要因素。当电流较小时，电池内阻的影响较为明显，导致电压较大幅度下降。但是随着电流增大，电池的内阻特性逐渐改变，可能表现为电池的温度升高、化学反应等因素发生变化，从而减缓了电压的下降幅度。换句话说，随着电流增加，电池内阻对电压的影响可能变得不那么敏感。

电池的极化效应：电池在放电过程中，尤其是在高电流放电时，会产生一定的极化现象。极化效应通常会导致电池电压在短时间内剧烈下降，特别是在电流增加到某一阈值时，电池内部的离子迁移和反应过程变得更加复杂。当电流增大到一定程度时，电池内部的反应速度可能无法进一步提高，因此电压下降的幅度会有所减小。

电池的温度效应：随着电流的增加，电池的温度会升高。温度的升高可能导致电池的内部化学反应加速，从而减缓电压的进一步下降。在一些情况下，高温会使得电池的电化学性能发生改变，导致电压的下降幅度逐渐变小。

电池容量和放电曲线：对于锂电池等化学反应型电池，电池的容量和放电曲线也会影响电压的下降速率。当电流从 1A 增加到 2A 时，电池的放电效率变化较大，电压会出现明显的下降。然而，继续增大电流时，电池内部的离子流动受阻，反应速率达到了饱和，电压下降的幅度可能会减小。

结论：

这种现象通常是由于电池的内阻、极化效应和温度效应等多方面因素的相互作用造成的。在一定电流区间内，电池的电压下降幅度会增大，但当电流过高时，由于电池的物理和化学特性（如反应速率饱和、温度效应等），电压的下降幅度会变得较小。这是一个较为复杂的非线性现象，涉及到电池的多种动态行为。

（2）充放电工作情况

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
```

```
opt = JuBat.Option()
```

```
# 充放电电流参数
charging_current = 4.0 # 充电电流（安培）
discharging_current = -2.0 # 放电电流（安培）
charge_time = 2000 # 充电时间（秒）
discharge_time = 2000 # 放电时间（秒）
total_time = charge_time + discharge_time # 总时间
```

```
# 脉冲电流参数
pulse_current = 3.0 # 峰值脉冲电流（安培）
low_current = 0.5 # 低电流（安培）
pulse_width = 200 # 脉冲宽度（秒）
pulse_interval = 1000 # 脉冲间隔（秒）
```

```
opt.dtType = "fixed"
opt.time = [0, total_time] # 设置总模拟时间
```

```
# 设置电流随时间变化：充电和放电阶段的脉冲电流
opt.Current = (t) -> begin
    if t < charge_time
        # 充电阶段的脉冲电流
        if t % (pulse_width + pulse_interval) < pulse_width
            return charging_current + pulse_current # 充电阶段加上脉冲电流
        else
            return charging_current + low_current # 充电阶段加上低电流
        end
    else
        # 放电阶段的脉冲电流
        if t % (pulse_width + pulse_interval) < pulse_width
            return discharging_current + pulse_current # 放电阶段加上脉冲电流
        else
            return discharging_current + low_current # 放电阶段加上低电流
        end
    end
end
```

```
# 运行模型模拟
for model in ["sP2D", "P2D"]
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

```
# 绘制结果
```

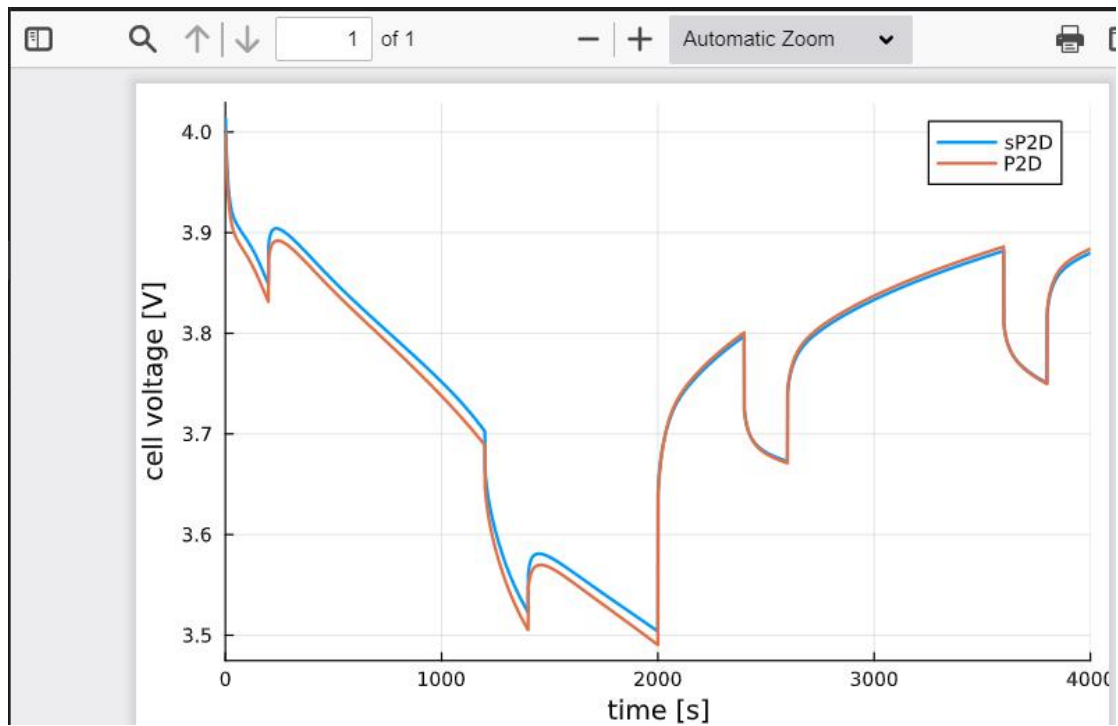
```
plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="time [s]",  
ylabel="cell voltage [V]", lw=2)  
end
```

```
# 设置 x 轴显示范围为 total_time  
xlims!(0, total_time)
```

```
# 保存图形  
savefig("pulse_charge_discharge_current_validation.pdf")
```

```
# 引用  
JuBat.Citation(["ai2024b"])
```

结果图：



四、交流电流 sin 工作环境（未完全成功）

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")
```

```
param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# 交流电流（AC）参数
amplitude = 2.0 # 电流幅度（安培）
frequency = 0.1 # 频率（Hz）
offset = 0.0 # 偏移量（安培），通常设为0
```

```
total_time = 5000 # 总模拟时间（秒）
```

```
opt.dtType = "fixed"
opt.time = [0, total_time] # 设置总模拟时间
```

```
# 设置为正弦波电流
opt.Current = (t) -> begin
    return amplitude * sin(2 * π * frequency * t) + offset
end
```

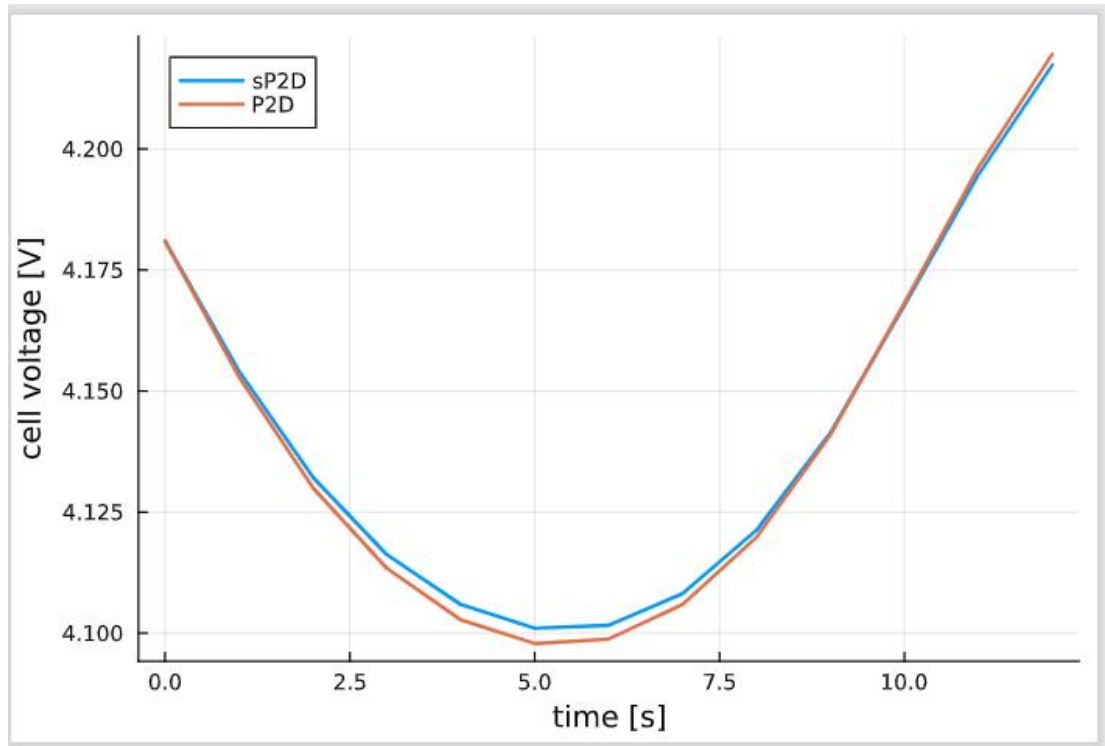
```
# 运行模型模拟
for model in ["sP2D", "P2D"]
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

```
# 绘制结果
plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="time [s]",
ylabel="cell voltage [V]", lw=2)
end
```

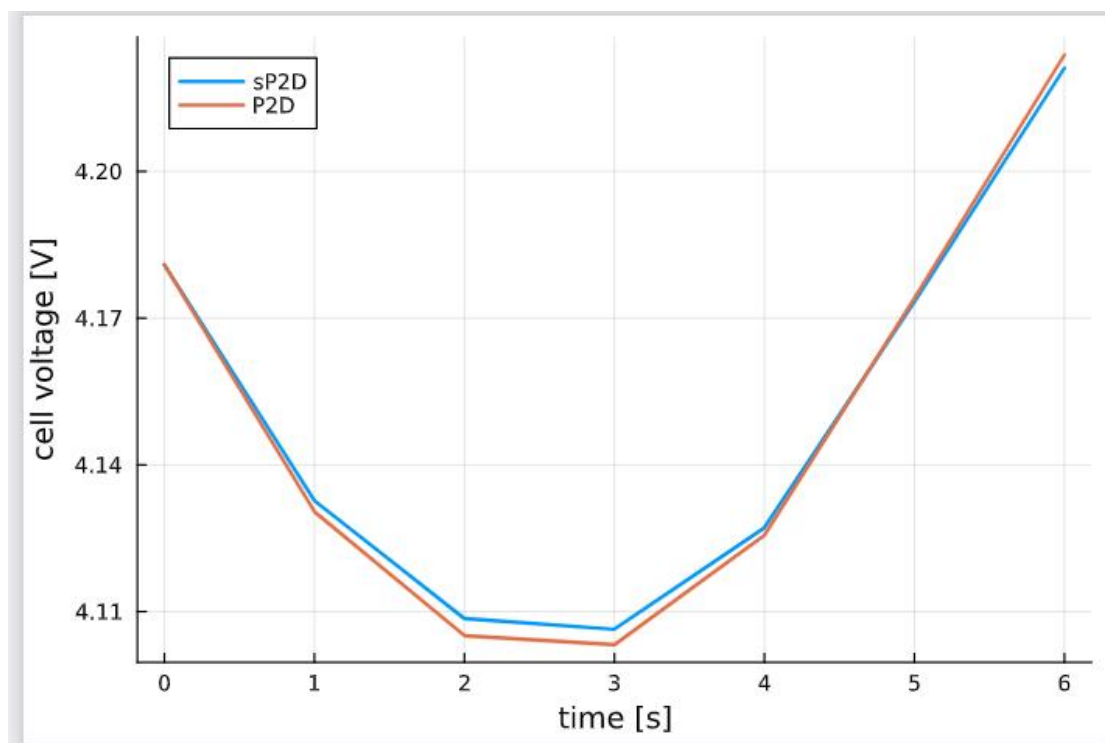
```
# 保存图形
savefig("ac_current_validation.pdf")
```

```
# 引用
JuBat.Citation(["ai2024b"])
```

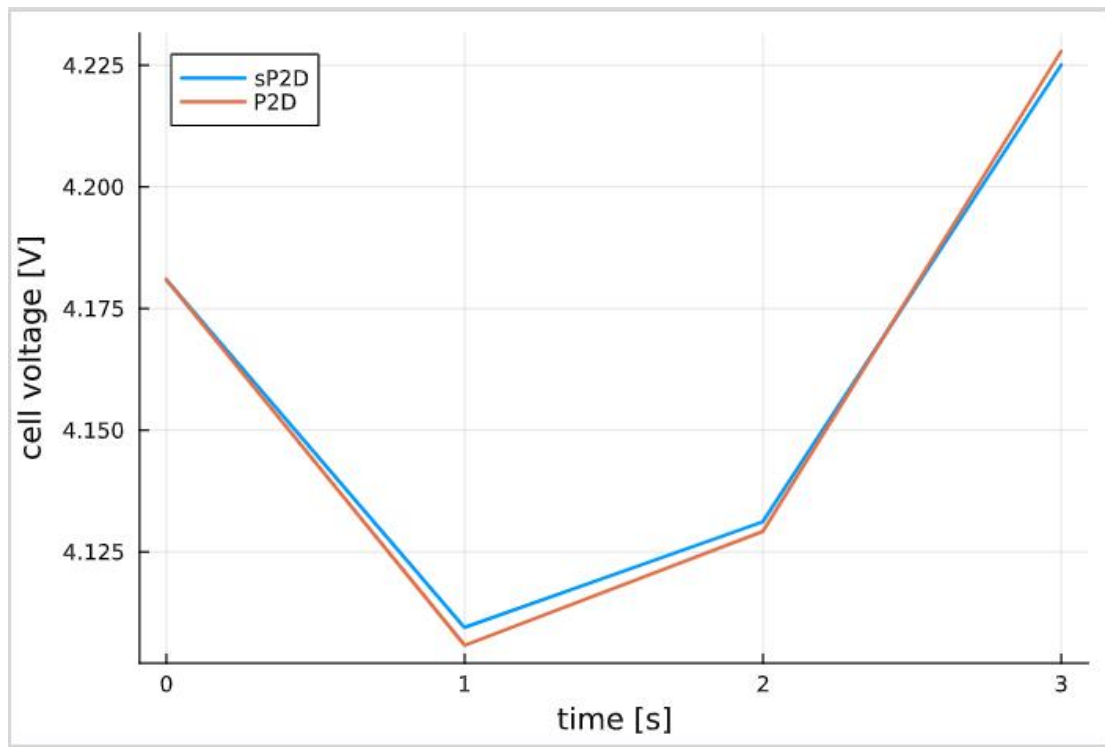
0.05HZ:



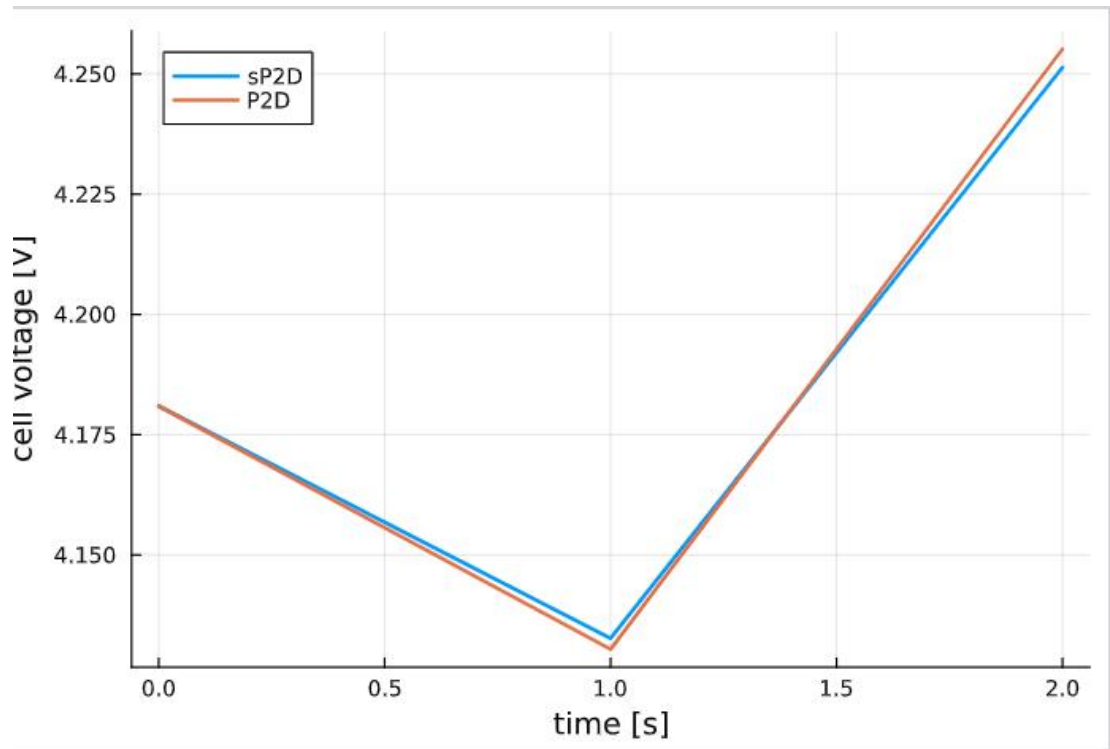
0.1HZ:



0.2HZ:



0.4HZ:



五、交流电流 linear 工作环境（成功）

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")
```

```
# 定义电池参数
param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# 设置阶梯电流工作条件下的电流参数
current_steps = [4.0, 2.0, 0.0, -2.0, -4.0] # 阶梯电流（安培）：充电阶段 -> 放电阶段
current_durations = [2000, 2000, 2000, 2000, 2000] # 每个电流阶段的持续时间（秒）
```

```
# 设置模拟时间
total_time = sum(current_durations) # 总时间
```

```
# 初始化模拟配置
opt.dtType = "fixed"
opt.time = [0, total_time] # 设置总模拟时间
```

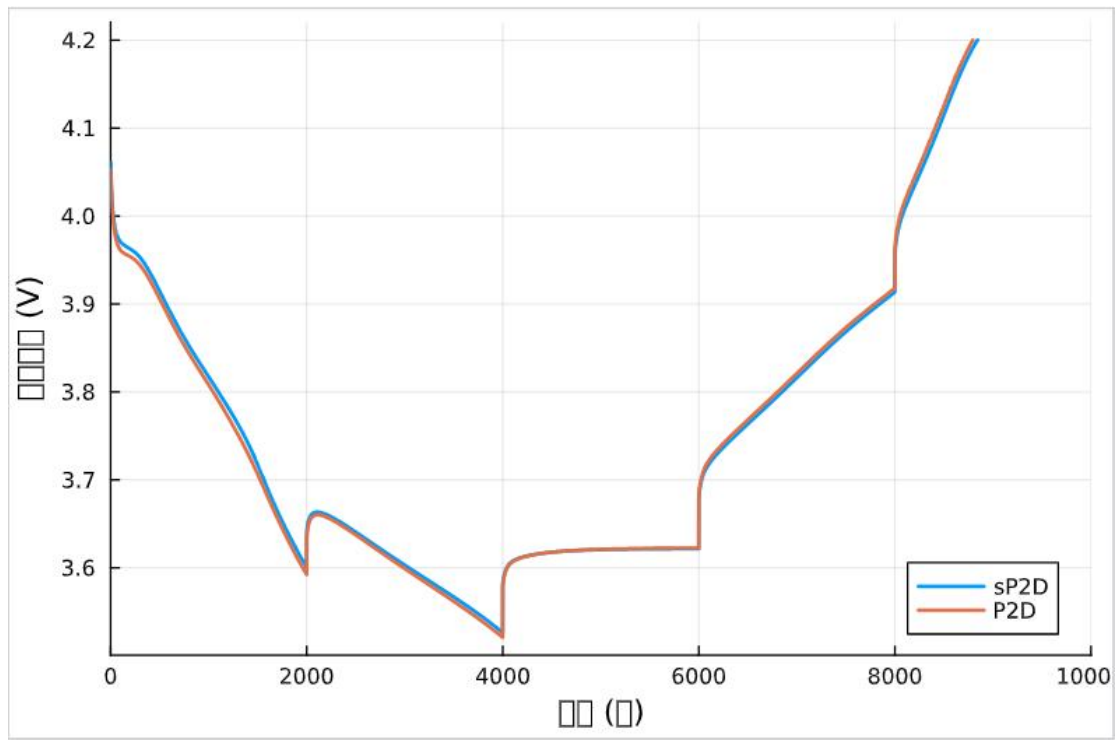
```
# 设置电流随时间的变化：阶梯电流充电与放电阶段
opt.Current = (t) -> begin
    # 计算当前时间所在的阶段
    elapsed_time = 0
    for (i, duration) in enumerate(current_durations)
        elapsed_time += duration
        if t <= elapsed_time
            return current_steps[i] # 返回当前阶段的电流
        end
    end
    return 0.0 # 默认情况下为零电流（理论上不会到达）
end
```

```
# 运行模型模拟并比较 P2D 和 SP2D 模型
for model in ["SP2D", "P2D"]
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

```
# 绘制模型结果
plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="时间（秒）", ylabel="
电池电压（V）", lw=2)
end
```

```
# 设置 x 轴显示范围为 total_time
xlims!(0, total_time)
```

```
# 保存图形
savefig("charge_discharge_step_comparison.pdf")
```



六、快速充放电电流工作条件

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# 快速充放电电流参数
charging_current = 8.0 # 充电电流 (安培)
discharging_current = -8.0 # 放电电流 (安培)
charge_time = 200 # 充电时间 (秒)
discharge_time = 200 # 放电时间 (秒)
total_time = charge_time + discharge_time # 总时间
```

```
opt.dtType = "fixed"
opt.time = [0, total_time] # 设置总模拟时间
```

```
# 设置快速充放电电流（充电阶段和放电阶段）
opt.Current = (t) -> begin
    if t < charge_time
        return charging_current # 充电阶段
    else
        return discharging_current # 放电阶段
    end
end
```

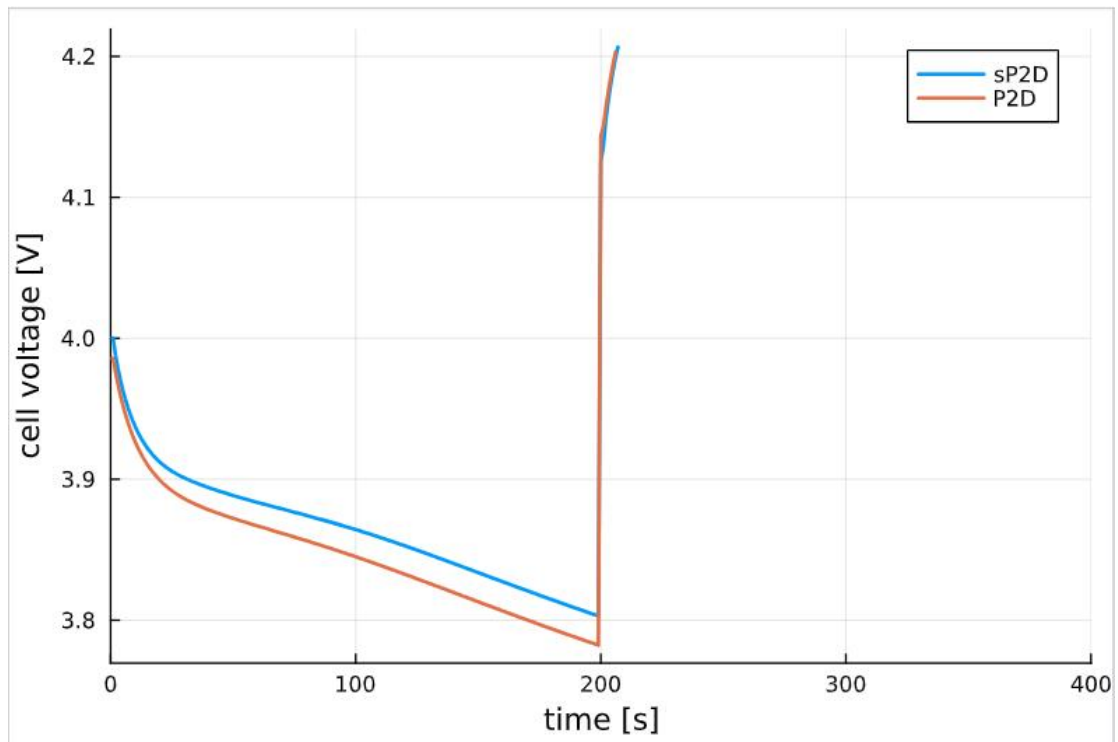
```
# 运行模型模拟
for model in ["sP2D", "P2D"]
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

```
# 绘制结果
plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="time [s]",
ylabel="cell voltage [V]", lw=2)
end
```

```
# 设置x轴显示范围为total_time
xlims!(0, total_time)
```

```
# 保存图形
savefig("fast_charge_discharge_current_validation.pdf")
```

```
# 引用
JuBat.Citation(["ai2024b"])
```



七、锯齿电流工作条件

(其中一个失败案例)

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# 锯齿波电流参数
max_current = 2.0 # 最大电流 (安培)
min_current = -2.0 # 最小电流 (安培)
period = 1000 # 一个周期的时间 (秒)
total_time = 5000 # 总模拟时间 (秒)
```

```
opt.dtType = "fixed"
opt.time = [0, total_time] # 设置总模拟时间
```

```
# 设置电流为锯齿波
opt.Current = (t) -> begin
    return min_current + (max_current - min_current) * (mod(t, period) / period)
end
```

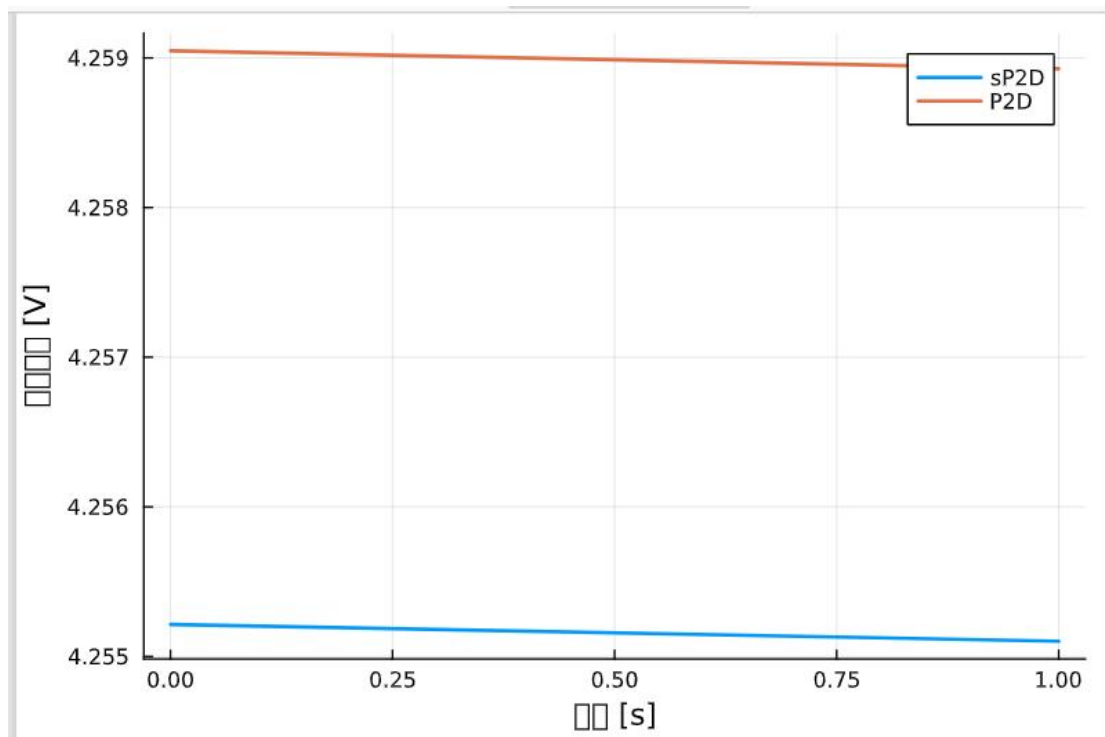
```
# 运行模型模拟
```

```
for model in ["sP2D", "P2D"]
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

```
# 绘制结果
plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="时间 [s]", ylabel="
电池电压 [V]", lw=2)
end
```

```
# 保存图形
savefig("sawtooth_current_validation.pdf")
```

```
# 引用
JuBat.Citation(["ai2024b"])
```



放电工作条件（从低到高）（成功）

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
```

```
opt = JuBat.Option()
```

```
# 定义锯齿波电流参数
```

```
amplitude = 3.0 # 锯齿波的幅值（安培）
```

```
period = 500 # 锯齿波的周期（秒）
```

```
total_time = 3000 # 总模拟时间（秒）
```

```
opt.dtType = "fixed"
```

```
opt.time = [0, total_time] # 设置总模拟时间
```

```
# 定义锯齿波电流函数
```

```
opt.Current = (t) -> amplitude * (t / period - floor(t / period))
```

```
# 运行模型模拟
```

```
for model in ["sP2D", "P2D"]
```

```
    opt.model = model
```

```
    case = JuBat.SetCase(param_dim, opt)
```

```
    result = JuBat.Solve(case)
```

```
# 绘制结果
```

```
    plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="time [s]",  
ylabel="cell voltage [V]", lw=2)
```

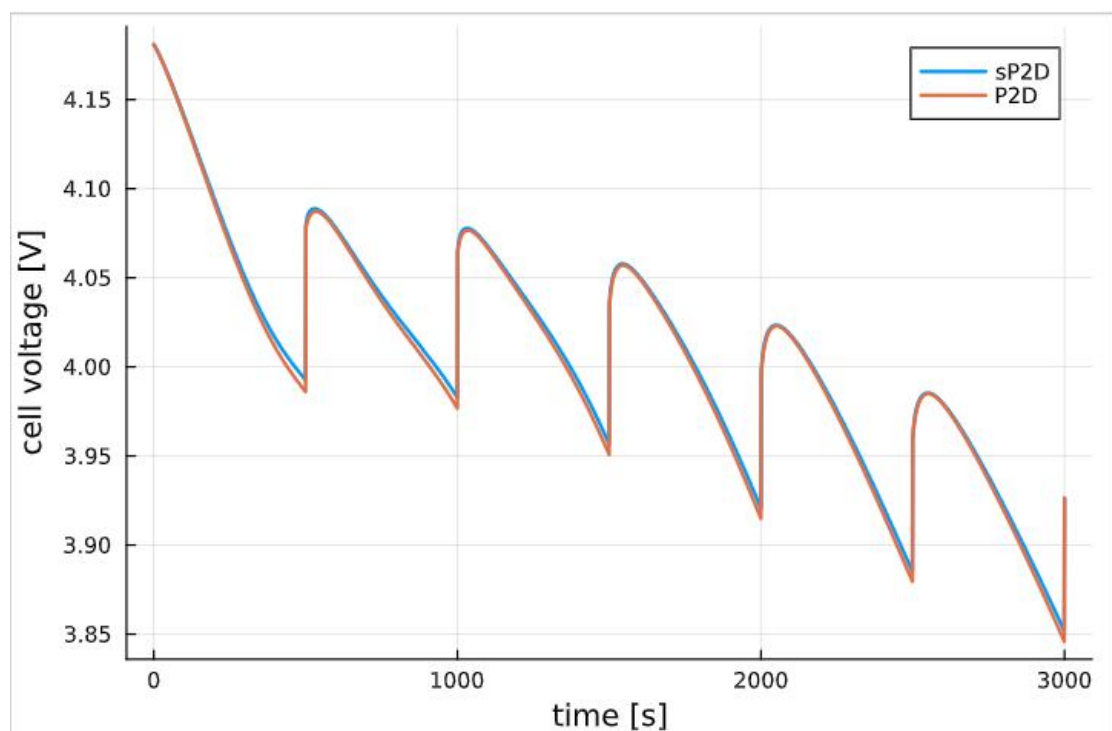
```
end
```

```
# 保存图形
```

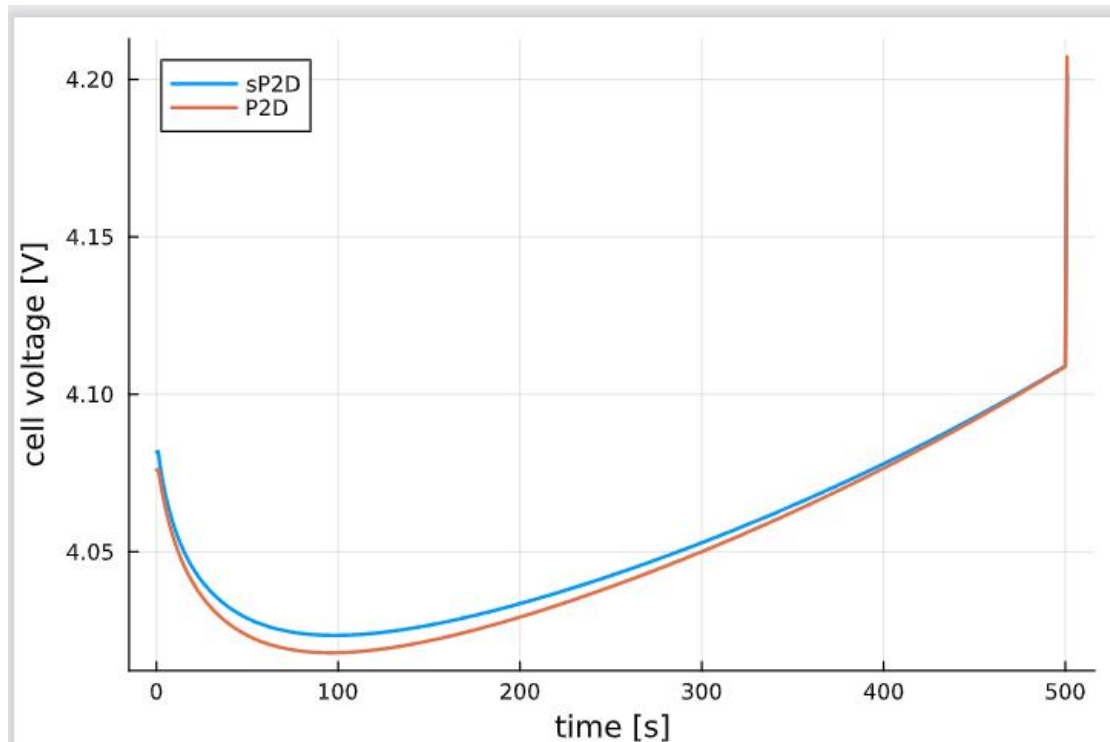
```
savefig("sawtooth_current_validation.pdf")
```

```
# 引用
```

```
JuBat.Citation(["ai2024b"])
```



放电工作条件（从高到低）（失败案例）



放电工作条件（从高到低）（成功）

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()

# 定义锯齿波电流参数
amplitude = 3.0 # 锯齿波的幅值（安培）
period = 500    # 锯齿波的周期（秒）
total_time = 3000 # 总模拟时间（秒）

opt.dtType = "fixed"
opt.time = [0, total_time] # 设置总模拟时间

# 定义下降锯齿波电流函数
opt.Current = (t) -> amplitude * (1 - (t % period) / period)

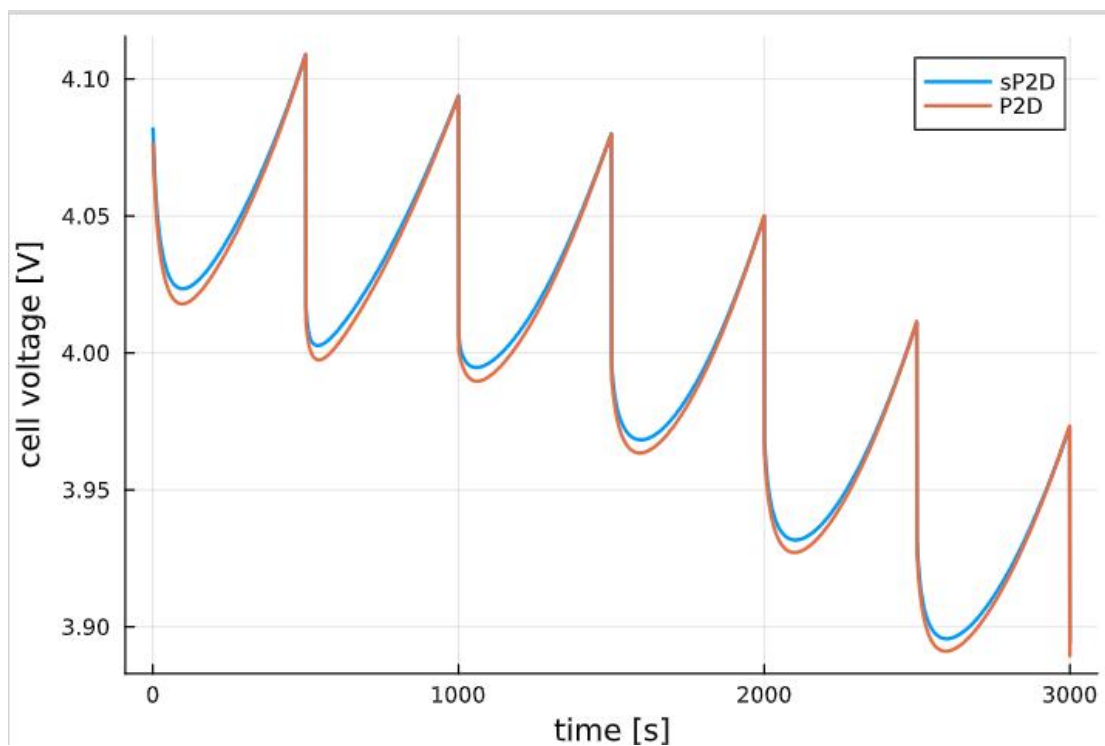
# 运行模型模拟
for model in ["sP2D", "P2D"]
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
```

```
result = JuBat.Solve(case)
```

```
# 绘制结果  
plot!(result["time [s]"], result["cell voltage [V]"], label=model, xlabel="time [s]",  
ylabel="cell voltage [V]", lw=2)  
end
```

```
# 保存图形  
savefig("falling_sawtooth_current_validation.pdf")
```

```
# 引用  
JuBat.Citation(["ai2024b"])
```



变化放电工作条件（先恒定电流放电，再锯齿电流放电）

```
using Plots, CSV, DataFrames  
include("../src/JuBat.jl")  
  
param_dim = JuBat.ChooseCell("LG M50")  
opt = JuBat.Option()
```

```
# 恒流放电参数  
charging_current = 4.0 # 放电电流 (A)  
charge_time = 2000 # 放电总时间 (秒)
```

```
# 锯齿波充电参数
discharge_amplitude = 4.0 # 充电峰值电流绝对值 (A)
discharge_period = 500    # 充电锯齿波周期 (秒)
discharge_time = 2000     # 充电总时间 (秒)
```

```
total_time = charge_time + discharge_time
```

```
opt.dtType = "fixed"
opt.time = [0, total_time]
```

```
# 定义复合电流函数
opt.Current = (t) -> begin
  if t < charge_time
    # 放电阶段: 恒定电流
    return charging_current
  else
    # 充电阶段: 周期性下降锯齿波
    return discharge_amplitude * (1 - (t % discharge_period) / discharge_period)
  end
end
```

```
# 初始化绘图并设置网格线
plt = plot(grid=true) # <-- 这里初始化时直接启用网格
```

```
# 运行模型比较
models = ["sP2D", "P2D"]
```

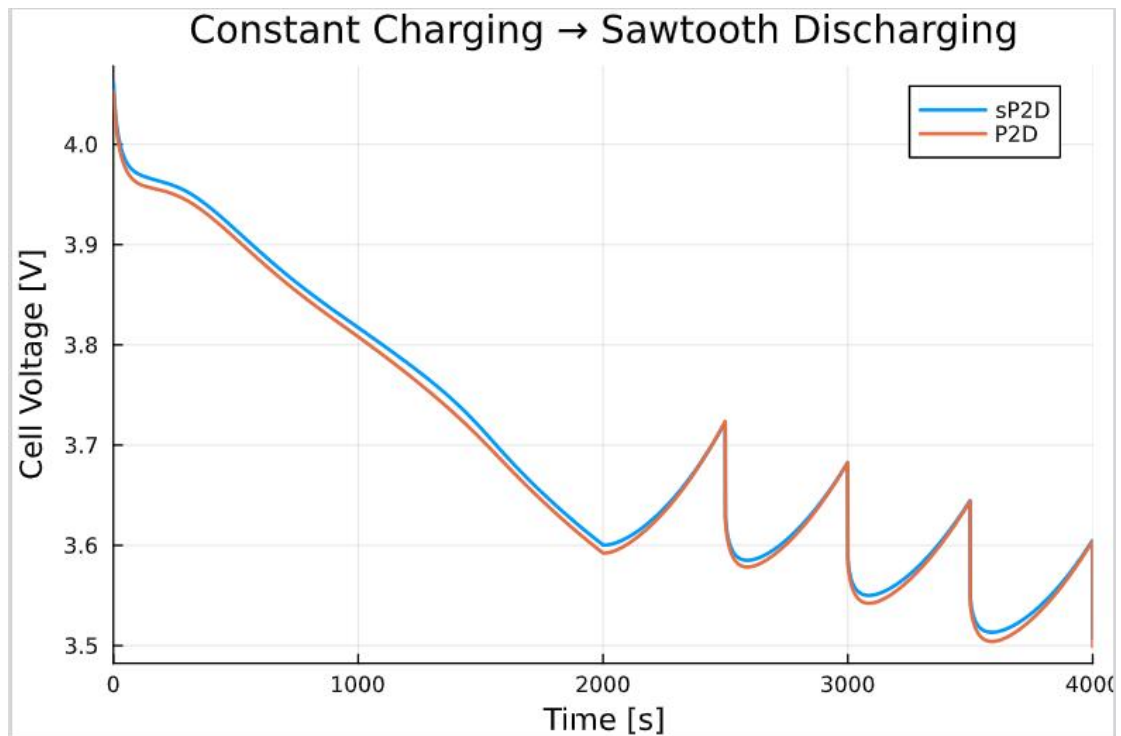
```
for model in models
  opt.model = model
  case = JuBat.SetCase(param_dim, opt)
  result = JuBat.Solve(case)

  # 绘图时保持网格设置
  plot!(plt, result["time [s]"], result["cell voltage [V]"],
    label=model, lw=2, xlabel="Time [s]", ylabel="Cell Voltage [V]",
    grid=true) # <-- 这里确保每次叠加都保持网格
end
```

```
# 图形修饰
xlims!(0, total_time)
title!("Constant DisCharging → Sawtooth Discharging")
savefig("constant_discharge then sawtooth_discharge.pdf")
```

```
# 引用
```

```
JuBat.Citation(["ai2024b"])
```



充放电工作条件（锯齿电流放电，恒定电流充电）

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# 锯齿波充电参数
charge_amplitude = 4.0 # 充电峰值电流 (A)
charge_period = 500 # 充电锯齿波周期 (秒)
charge_time = 2000 # 充电总时间 (秒)
```

```
# 恒流放电参数
discharge_current = -2.0 # 放电电流 (A)
discharge_time = 2000 # 放电总时间 (秒)
```

```
total_time = charge_time + discharge_time
```

```
opt.dtType = "fixed"
opt.time = [0, total_time]
```

```
# 定义复合电流函数
```

```

opt.Current = (t) -> begin
  if t < charge_time
    # 充电阶段：周期性下降锯齿波
    return charge_amplitude * (1 - (t % charge_period) / charge_period)
  else
    # 放电阶段：恒定电流
    return discharge_current
  end
end
end

```

```

# 初始化绘图并设置网格线
plt = plot(grid=true) # <-- 这里初始化时直接启用网格

```

```

# 运行模型比较
models = ["sP2D", "P2D"]

```

```

for model in models
  opt.model = model
  case = JuBat.SetCase(param_dim, opt)
  result = JuBat.Solve(case)

  # 绘图时保持网格设置
  plot!(plt, result["time [s]"], result["cell voltage [V]"],
    label=model, lw=2, xlabel="Time [s]", ylabel="Cell Voltage [V]",
    grid=true) # <-- 这里确保每次叠加都保持网格
end

```

```

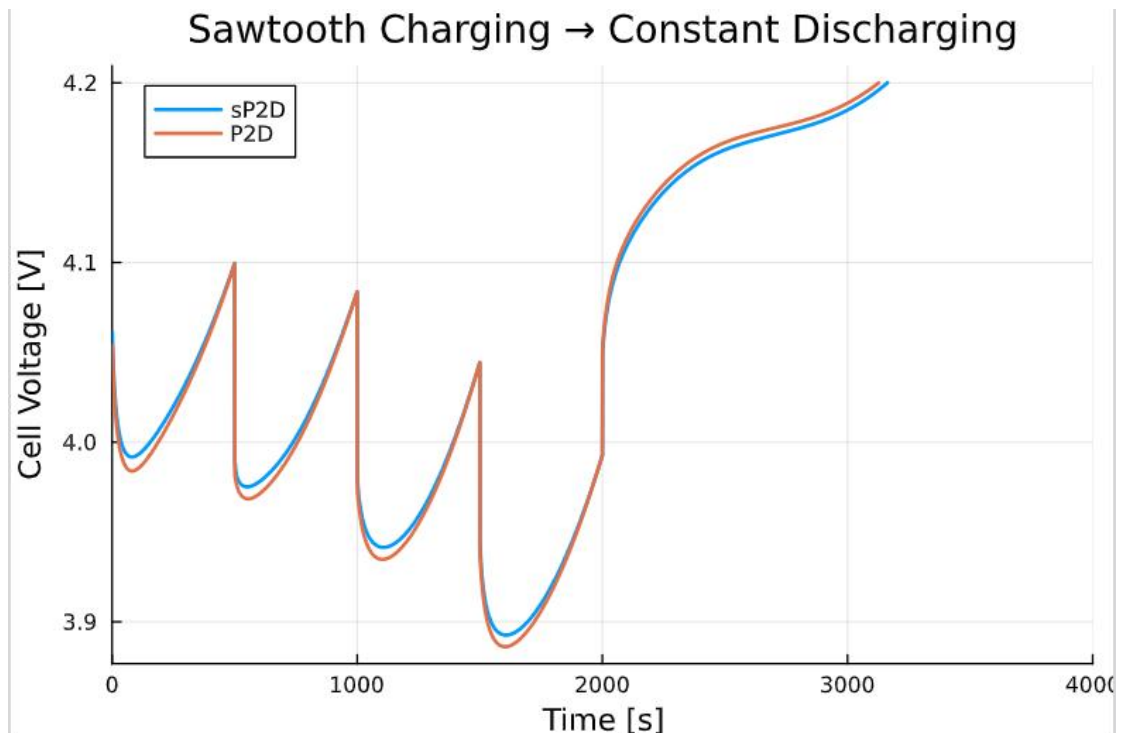
# 图形修饰
xlims!(0, total_time)
title!("Sawtooth Charging → Constant Discharging")
savefig("sawtooth_charge_constant_discharge.pdf")

```

```

# 引用
JuBat.Citation(["ai2024b"])

```



充放电工作条件（恒定电流放电，锯齿电流充电）

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")

param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# 恒流放电参数
charging_current = 4.0 # 放电电流 (A)
charge_time = 2000 # 放电总时间 (秒)
```

```
# 锯齿波充电参数
discharge_amplitude = -4.0 # 充电峰值电流绝对值 (A)
discharge_period = 500 # 充电锯齿波周期 (秒)
discharge_time = 2000 # 充电总时间 (秒)
```

```
total_time = charge_time + discharge_time
```

```
opt.dType = "fixed"
opt.time = [0, total_time]
```

```
# 定义复合电流函数
opt.Current = (t) -> begin
```

```

    if t < charge_time
        # 放电阶段: 恒定电流
        return charging_current
    else
        # 充电阶段: 周期性下降锯齿波
        return discharge_amplitude * (1 - (t % discharge_period) / discharge_period)
    end
end

```

```

# 初始化绘图并设置网格线
plt = plot(grid=true) # <-- 这里初始化时直接启用网格

```

```

# 运行模型比较
models = ["sP2D", "P2D"]

```

```

for model in models
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)

    # 绘图时保持网格设置
    plot!(plt, result["time [s]"], result["cell voltage [V]"],
        label=model, lw=2, xlabel="Time [s]", ylabel="Cell Voltage [V]",
        grid=true) # <-- 这里确保每次叠加都保持网格
end

```

```

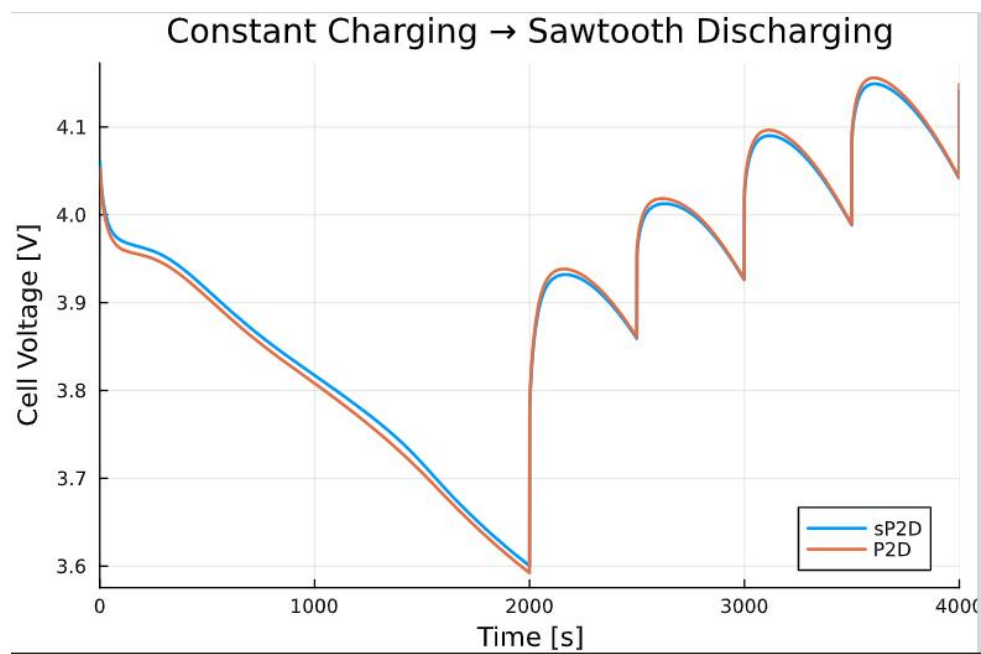
# 图形修饰
xlims!(0, total_time)
title!("Constant Charging → Sawtooth Discharging")
savefig("constant_charge_sawtooth_discharge.pdf")

```

```

# 引用
JuBat.Citation(["ai2024b"])

```



充放电工作条件（锯齿电流放电，锯齿电流充电）（高到低）

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")
```

```
param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# 锯齿波流放电参数
```

```
charge_amplitude = 4.0 # 放电电流 (A)
charge_period = 500 # 放电锯齿波周期 (秒)
charge_time = 2000 # 放电总时间 (秒)
```

```
# 锯齿波充电参数
```

```
discharge_amplitude = -2.0 # 充电峰值电流绝对值 (A)
discharge_period = 500 # 充电锯齿波周期 (秒)
discharge_time = 2000 # 充电总时间 (秒)
```

```
total_time = charge_time + discharge_time
```

```
opt.dtType = "fixed"
opt.time = [0, total_time]
```

```
# 定义复合电流函数
```

```
opt.Current = (t) -> begin
    if t < charge_time
        # 放电阶段：周期性下降锯齿波
        return charge_amplitude * (1 - (t % charge_period) / charge_period)
    else
        # 充电阶段：周期性下降锯齿波
        return discharge_amplitude * (1 - (t % discharge_period) / discharge_period)
    end
end
```

```
# 初始化绘图并设置网格线
```

```
plt = plot(grid=true) # <-- 这里初始化时直接启用网格
```

```
# 运行模型比较
```

```
models = ["sP2D", "P2D"]
```

```
for model in models
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

```

# 绘图时保持网格设置
plot!(plt, result["time [s]"], result["cell voltage [V]"],
      label=model, lw=2, xlabel="Time [s]", ylabel="Cell Voltage [V]",
      grid=true) # <-- 这里确保每次叠加都保持网格
end

```

```

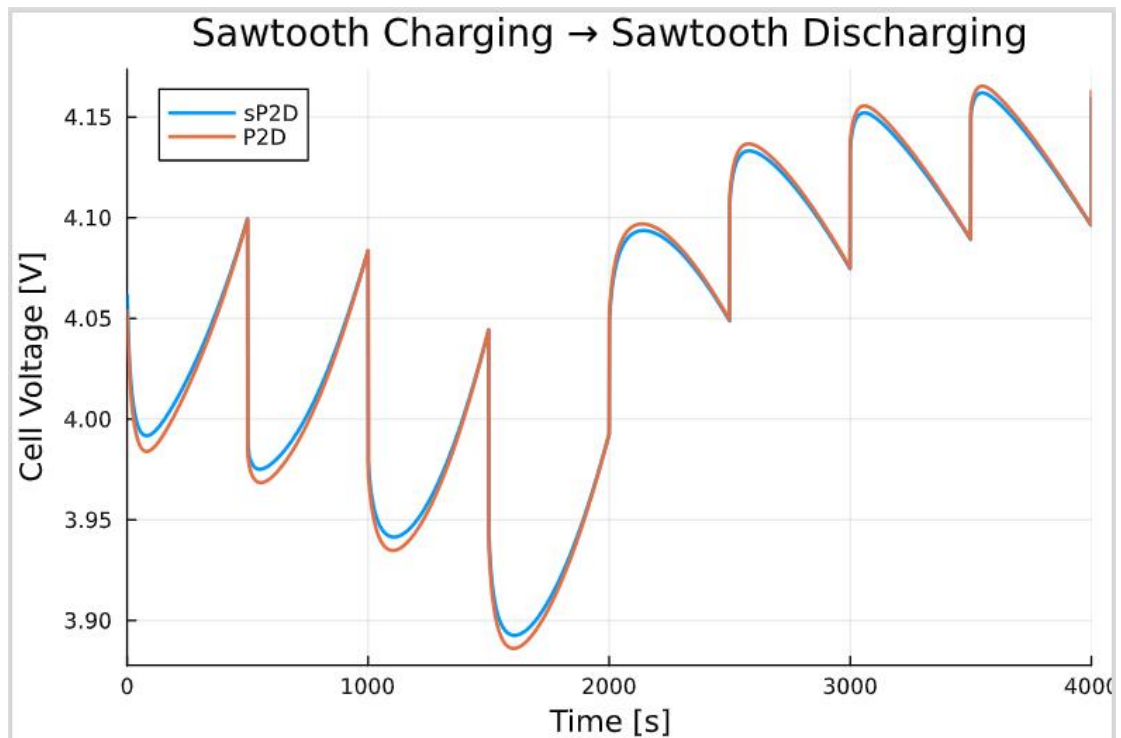
# 图形修饰
xlims!(0, total_time)
title!("Sawtooth Charging → Sawtooth Discharging")
savefig("Sawtooth_charge_sawtooth_discharge.pdf")

```

```

# 引用
JuBat.Citation(["ai2024b"])

```



充放电工作条件（锯齿电流放电，锯齿电流充电）（低到高）

```
using Plots, CSV, DataFrames
include("../src/JuBat.jl")
```

```
param_dim = JuBat.ChooseCell("LG M50")
opt = JuBat.Option()
```

```
# 锯齿波流放电参数
```

```
charge_amplitude = 4.0 # 放电电流 (A)
charge_period = 500 # 放电锯齿波周期 (秒)
charge_time = 2000 # 放电总时间 (秒)
```

```
# 锯齿波充电参数
```

```
discharge_amplitude = -2.0 # 充电峰值电流绝对值 (A)
discharge_period = 500 # 充电锯齿波周期 (秒)
discharge_time = 2000 # 充电总时间 (秒)
```

```
total_time = charge_time + discharge_time
```

```
opt.dtType = "fixed"
opt.time = [0, total_time]
```

```
# 定义复合电流函数
```

```
opt.Current = (t) -> begin
    if t < charge_time
        # 放电阶段：周期性上升锯齿波
        return charge_amplitude * ((t % charge_period) / charge_period)
    else
        # 充电阶段：周期性上升锯齿波
        return discharge_amplitude * ((t % discharge_period) / discharge_period)
    end
end
```

```
# 初始化绘图并设置网格线
```

```
plt = plot(grid=true) # <-- 这里初始化时直接启用网格
```

```
# 运行模型比较
```

```
models = ["sP2D", "P2D"]
```

```
for model in models
    opt.model = model
    case = JuBat.SetCase(param_dim, opt)
    result = JuBat.Solve(case)
```

```

# 绘图时保持网格设置
plot!(plt, result["time [s]"], result["cell voltage [V]"],
      label=model, lw=2, xlabel="Time [s]", ylabel="Cell Voltage [V]",
      grid=true) # <-- 这里确保每次叠加都保持网格
end

```

```

# 图形修饰
xlims!(0, total_time)
title!("Sawtooth Charging → Sawtooth Discharging(up)")
savefig("Sawtooth_charge_sawtooth_discharge(low to high).pdf")

```

```

# 引用
JuBat.Citation(["ai2024b"])

```

